

Machine Learning

Consolidated Practice Question Bank

Assembled from ten past papers, 2016–2021

About this document. Every question and solution from the ten solution PDFs in `PracticeProblems/` appears exactly once below. Seventeen questions that were asked twice (or three times) across the papers have been merged; where the two source keys answered differently, both answers are kept. The grey line under each question number gives its provenance. Point values are inherited from the source papers and do not sum to a meaningful total.

Solutions are currently SHOWN. Toggle the `\printanswers` line in the preamble.

Contents

Part I	Convexity and Optimization	2
Part II	Linear Regression and Regularization	7
Part III	Logistic Regression and Classification Losses	12
Part IV	Naive Bayes and Generative Classifiers	18
Part V	Nearest Neighbours and Instance-Based Methods	21
Part VI	Support Vector Machines and Kernels	23
Part VII	Decision Trees and Ensembles	32
Part VIII	Neural Networks and Deep Learning	36
Part IX	Unsupervised Learning and Dimensionality Reduction	47
Part X	Model Evaluation, Bias–Variance and Cross-Validation	49
Part XI	Sequential Decision Making	52
Part XII	True/False and Multiple-Choice Blocks	53

Part I Convexity and Optimization

I.1. (10 points) [QUIZ2 Q1 $\equiv$ END2018 Q5(a)]

(10=3+3+4) Define a convex set and a convex function mathematically. What is a convex optimization problem?

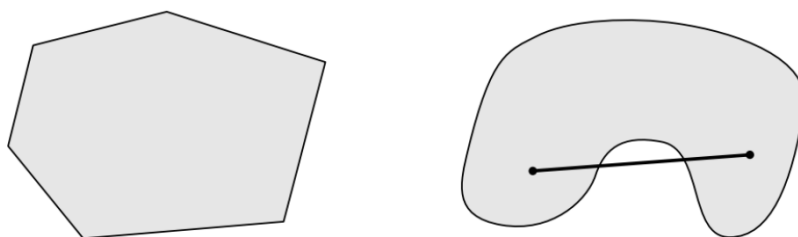
Solution:

Convex Sets

A set C is convex if, for any $x, y \in C$ and $\theta \in \mathbb{R}$ with $0 \leq \theta \leq 1$,

$$\theta x + (1 - \theta)y \in C$$

If we take any two elements in C , and draw a line segment between these two elements, then every point on that line segment also belongs to C . The point $\theta x + (1 - \theta)y$ is convex combination of points x and y .

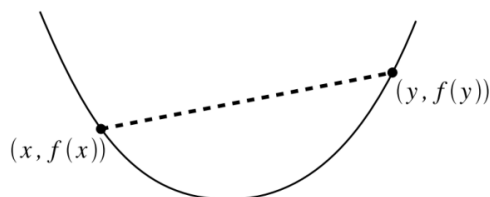


Examples of a convex set (a) and a non-convex set (b).

Convex Functions

A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if its domain $D(f)$ is a convex set, and if, for all $x, y \in D(f)$ and $\theta \in \mathbb{R}, 0 \leq \theta \leq 1$,

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$



If a straight line is drawn between any two points on the graph of a convex function, then the portion of a function between these two points will lie below the straight line. The line connecting the two points will lie above the function.

Convex Optimization

Convex optimization problem is an optimization problem of the form

$$\begin{aligned} & \text{minimize } f(x) \\ & \text{subject to } g_i(x) \leq 0, \quad i = 1, \dots, m \\ & \quad \quad h_i(x) = 0, \quad i = 1, \dots, p \end{aligned}$$

where f is a convex function, g_i are convex functions, and h_i are affine functions, and x is the optimization variable.

A convex function g_i must be less than zero else the region will no longer be a convex set. Only affine functions are allowed to be equality constraints.

The optimal value p^* of an optimization problem is the minimum possible value of the objective function in the feasible region.

$$p^* = \min f(x) : g_i(x) \leq 0, i = 1, \dots, m, h_i(x) = 0, i = 1, \dots, p$$

p^* is allowed to take on the values $+\infty$ and $-\infty$ when the problem is 'infeasible' or 'unbounded below'. x^* is the optimal point if $f(x^*) = p^*$. There can be more than one optimal point, even when the optimal value is finite.

Reference: [link](#)

Alternative solution (END2018 Q5(a)): A set $\mathcal{S}$ is said to be convex if $\forall x_1, x_2 \in \mathcal{S}$, and $0 \leq \alpha \leq 1$, every point given by $y = (\alpha x_1 + (1 - \alpha)x_2) \in \mathcal{S}$. A function $f(x)$ is convex if for $0 \leq \alpha \leq 1$ and for all x and y the following property is satisfied: $f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$.

I.2. [QUIZ3 Q3 $\equiv$ END2018 Q5(a)]

(Extra Credit) Building upon the previous question, show that sum of any two convex functions is convex. Extend your argument to show that sum of any number of convex functions is convex.

Solution: Consider two convex functions f_1 and f_2 such that both are convex.

$$f_1(cx_1 + (1 - c)x_2) \leq cf_1(x_1) + (1 - c)f_1(x_2) \quad (1)$$

$$f_2(cx_1 + (1 - c)x_2) \leq cf_2(x_1) + (1 - c)f_2(x_2) \quad (2)$$

Now, add (1) and (2)

$$f_1(cx_1 + (1 - c)x_2) + f_2(cx_1 + (1 - c)x_2) \leq cf_1(x_1) + (1 - c)f_1(x_2) + cf_2(x_1) + (1 - c)f_2(x_2)$$

$$f_1(cx_1 + (1 - c)x_2) + f_2(cx_1 + (1 - c)x_2) \leq c(f_1(x_1) + f_2(x_1)) + (1 - c)(f_1(x_2) + f_2(x_2)) \quad (3)$$

Let $h = f_1 + f_2$.

We can now rewrite equation (3) as

$$h(cx_1 + (1 - c)x_2) \leq ch(x_1) + (1 - c)h(x_2)$$

This is convex as well, thus proving that the sum of two convex functions is also convex.

It is trivial to extend this to n functions. Assume you have n convex functions $f_1, f_2, f_3, \dots, f_n$. Now let $h = f_1 + f_2 + f_3 + \dots + f_n$

Similar to what was done with equations (1) and (2), we can add all n equations in this case to get an equation similar to that of equation (3)

$$f_1(cx_1 + (1-c)x_2) + f_2(cx_1 + (1-c)x_2) + \dots + f_n(cx_1 + (1-c)x_2) \leq \\ c(f_1(x_1) + f_2(x_1) + \dots + f_n(x_1)) + (1-c)(f_1(x_2) + f_2(x_2) + \dots + f_n(x_2))$$

We can now rewrite this as

$$h(cx_1 + (1-c)x_2) \leq ch(x_1) + (1-c)h(x_2)$$

This is convex as well, thus proving that the sum of n convex functions is also convex.

Alternative solution (END2018 Q5(a)): If $f(x)$ and $g(x)$ are both convex, then $h(x) = f(x) + g(x)$ will be convex as shown below

$$\begin{aligned} h(\alpha x + (1-\alpha)y) &= f(\alpha x + (1-\alpha)y) + g(\alpha x + (1-\alpha)y) \\ &\leq (\alpha f(x) + (1-\alpha)f(y)) + (\alpha g(x) + (1-\alpha)g(y)) \\ &\leq \alpha(f(x) + g(x)) + (1-\alpha)(f(y) + g(y)) \\ &\leq \alpha h(x) + (1-\alpha)h(y) \end{aligned}$$

I.3. (3 points) [END2018 Q5(b)]

Show that the loss functions for linear regression, ridge regression and LASSO are all convex functions with respect to the parameters.

Solution: Linear Regression: the loss is quadratic (convex), of the form $L(w) = \sum_{i=1}^N (y_i - w^\top x_i)^2$. Ridge regression has an additional quadratic regularization term in the parameters, i.e., $R(w) = \|w\|_2^2$, which is also convex. Finally LASSO has a regularization term $R(w) = \|w\|_1$, which is also a convex term.

I.4. [QUIZ4 Q1 $\equiv$ QUIZ3 Q1]

The binary cross-entropy loss is usually used with neural networks trained for a binary classification task. Write the mathematical form of the loss. Is it a convex function? Prove your claim.

Solution: Convex Function

A function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if its domain $D(f)$ is a convex set, and if, for all $x, y \in D(f)$ and $\theta \in \mathbb{R}, 0 \leq \theta \leq 1$,

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y)$$

First order condition of convexity: A function $f(x)$ which is differentiable is convex if the following inequality condition holds true:

For all $x, y \in D(f)$ and $\theta \in \mathbb{R}, 0 \leq \theta \leq 1$

$$f(y) \geq f(x) + \Delta_x^T f(x)(y - x)$$

Intuitively, this condition says that the tangent/first-order-taylor-series approximation of $f(x)$ is globally an under-estimator.

Second-order condition of convexity: A function $f(x)$ which is twice-differentiable is convex if and only if its hessian matrix (matrix of second-order partial derivatives) is positive semi-definite, i.e.

$$\forall z : z^T \Delta_x^2 f(x) z \geq 0$$

where $\Delta_x^2 f(x)$ is the hessian.

w.k.t linear combination of convex functions is convex.

$$(\lambda_1 f + \lambda_2 g)(x) = \lambda_1 f(x) + \lambda_2 g(x)$$

Proof: Cross Entropy Loss:

$$J(\theta) = \sum_{i=1}^m y^i [-\log(h_\theta(x^i))] + (1 - y^i) [-\log(1 - h_\theta(x^i))]$$

Now to prove $J(\theta)$ is convex we have to prove that the following two functions are convex.

$$-\log(h_\theta(x^i)) \quad \text{and} \quad -\log(1 - h_\theta(x^i))$$

Assume h_θ to be sigmoid.

function 1

$$-\log(h_\theta(x)) = -\log\left(\frac{1}{1 + e^{-\theta^T x}}\right) = \log(1 + e^{-\theta^T x})$$

To prove that the above function is convex, we will use the second-order condition of convexity described. Compute the hessian matrix:

grad :

$$\begin{aligned} \nabla_\theta [-\log(h_\theta(x))] &= \nabla_\theta [\log(1 + e^{-\theta^T x})] \\ &= \left(\frac{-e^{-\theta^T x}}{1 + e^{-\theta^T x}} \right) x \\ &= \left(\frac{1}{1 + e^{-\theta^T x}} - 1 \right) x \\ &= (h_\theta(x) - 1) x \end{aligned}$$

hessian :

$$\begin{aligned} \nabla_\theta^2 [-\log(h_\theta(x))] &= \nabla_\theta (\nabla_\theta [-\log(h_\theta(x))]) \\ &= \nabla_\theta ((h_\theta(x) - 1) x) \\ &= h_\theta(x) (1 - h_\theta(x)) x x^T \end{aligned}$$

Proof that this hessian matrix is positive semi-definite:

$$\begin{aligned} \forall z : z^T \nabla_x^2 (-\log(h_\theta(x))) &= z^T [h_\theta(x) (1 - h_\theta(x)) x x^T] z \\ &= h_\theta(x) (1 - h_\theta(x)) (x^T z)^2 \geq 0 \end{aligned}$$

function 2

$$\begin{aligned}
-\log(1 - h_\theta(x)) &= -\log\left(1 - \frac{1}{1 + e^{-\theta^T x}}\right) \\
&= -\log\left(\frac{e^{-\theta^T x}}{1 + e^{-\theta^T x}}\right) \\
&= \theta^T x + \log(1 + e^{-\theta^T x})
\end{aligned}$$

We will again use the second-order condition of convexity described. Compute the hessian matrix:

grad :

$$\begin{aligned}
\nabla_\theta [-\log(1 - h_\theta(x))] &= \nabla_\theta [\theta^T x + \log(1 + e^{-\theta^T x})] \\
&= x + \nabla_\theta [\log(1 + e^{-\theta^T x})]
\end{aligned}$$

hessian :

$$\begin{aligned}
\nabla_\theta^2 [-\log(1 - h_\theta(x))] &= \nabla_\theta (\nabla_\theta [-\log(1 - h_\theta(x))]) \\
&= \nabla_\theta (x + \nabla_\theta [\log(1 + e^{-\theta^T x})]) \\
&= \nabla_\theta^2 [-\log(h_\theta(x))]
\end{aligned}$$

We have already proved that $-\log(h_\theta(x))$ is positive semi-definite

As we have proved that both the functions are convex, $J(\theta)$ becomes convex (Linear combination of convex functions are convex).

Alternative solution (QUIZ3 Q1 – stated for the log-likelihood rather than the loss): The log-likelihood for logistic regression is given by $\mathcal{L}(\mathbf{w}) = y\mathbf{w}^T \mathbf{x} - \log(1 + \exp(\mathbf{w}^T \mathbf{x}))$. Using the definition of convexity for proving is tricky because of the addition within the log term. So we look at the second order condition for convexity, i.e., we compute the Hessian. The bias term here is assumed to be included in $\mathbf{w}$ and accordingly a 1 is appended to the data sample $\mathbf{x}$, and $\sigma(z)$ is the sigmoid function, the derivative of which is $\sigma(z)(1 - \sigma(z))$.

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= y\mathbf{x} - \mathbf{x} \left(\frac{1}{1 + e^{\mathbf{w}^T \mathbf{x}}} \right) (e^{\mathbf{w}^T \mathbf{x}}) \\
&= y\mathbf{x} - \mathbf{x} \left(\frac{e^{\mathbf{w}^T \mathbf{x}}}{1 + e^{\mathbf{w}^T \mathbf{x}}} \right) \\
&= y\mathbf{x} - \mathbf{x} \left(\frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}} \right) \\
&= y\mathbf{x} - \mathbf{x} (\sigma(\mathbf{w}^T \mathbf{x})) \\
\frac{\partial^2 \mathcal{L}}{\partial \mathbf{w} \mathbf{w}^T} &= 0 - \mathbf{x} \left(\sigma(\mathbf{w}^T \mathbf{x}) (1 - \sigma(\mathbf{w}^T \mathbf{x})) \frac{\partial \mathbf{w}^T \mathbf{x}}{\partial \mathbf{w}^T} \right) \\
&= - \left(\sigma(\mathbf{w}^T \mathbf{x}) (1 - \sigma(\mathbf{w}^T \mathbf{x})) \right) \mathbf{x} \mathbf{x}^T
\end{aligned}$$

Since $\sigma(z) \geq 0$, and $\mathbf{x} \mathbf{x}^T$ is a quadratic form (positive semi-definite), the Hessian is a positive semi-definite matrix multiplied by a negative number. This implies the Hessian is negative semi-definite. Therefore, technically, the log-likelihood function for logistic regression is not convex, but it is *concave* – equivalently, its negative (the loss proved convex above) is convex.

I.5. (15 points) [QUIZ2 Q3]

Given the following optimization problem, answer the questions below

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & x_1^2 + x_2 \\ \text{subject to} \quad & 2x_1 + x_2 \geq 4 \\ & x_2 \geq 1 \end{aligned}$$

- (a) (5 points) Show that this is a convex optimization problem.
- (b) (4 points) Graphically indicate the *feasible region* of the solution within $\mathbb{R}^2$.
- (c) (6 points) Write the Lagrangian for the above optimization problem. Make sure you specify the range of the Lagrange variables.

Solution: Refer Section 2, A simple duality example in the [link](#).

I.6. (10 points) [END2018 Q5(c)]

Using a simple (the simplest that you can come up with) neural network, show that learning the weights for a squared error loss term is a non-convex problem.

Solution: [The source PDF prints the label “(c)” with no answer – no solution was written for this part.]

Part II Linear Regression and Regularization

II.1. (20 points) [MID2018 Q2]

Consider the following simple linear regression model.

$$y = wx + \epsilon$$

Where, y is the sum of deterministic linear function of x and some noise ϵ . x and y are the real-valued input and output respectively, w is the real-valued parameter to be learned. $\epsilon \sim N(0, \sigma)$ is a Gaussian random variable representing the noise.

- (a) (5 points) Write the expression for the probability distribution of y (i.e $p(y|w, x)$) in terms of $N(\cdot, \cdot)$, w , σ , x .
- (b) (5 points) Suppose, we are given m i.i.d training samples $\{(y^1, x^1), \dots, (y^m, x^m)\}$, $\mathcal{Y} = \{y^1, \dots, y^m\}$ and $\mathcal{X} = \{x^1, \dots, x^m\}$. Derive an expression for conditional data likelihood (i.e $P(\mathcal{Y}|\mathcal{X}, w)$).
- (c) (5 points) Consider the conditional data likelihood, derive an expression for gradient descent learning rule.
- (d) (5 points) Suppose we are given a Laplacian prior ($p(x) = e^{-\lambda|x|}$, $x \in \mathbb{R}, \lambda > 0$) over w . What kind of regularization does it impose on your regular regression problem? Can you derive the expression for the log posterior? {Hint: Start with MLE, use Bayes rule and apply log, simple!}

Solution:

- (a)
- $y = wx + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma)$
- (1 mark)

Therefore y follows the distribution

$$p(y|x; w) = \mathcal{N}(f(x), \sigma), \text{ where } f(x) = wx$$

$$p(y|x; w) = \mathcal{N}(w_0 + \sum w_i x_i, \sigma) \text{ (2marks)}$$

$$p(y|x; w) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{y-f(x)}{\sigma}\right)^2} \text{ (2marks)}$$

- (b) From the above expression

$$p(y|x; w) = p(y_1, y_2, y_3, y_4 \dots | \theta) \prod_i p(y_i | \theta) \text{ (2marks)}$$

$$\ln p(y|x; w) = \sum \ln p(y_i | \theta)$$

$$(1\text{mark}) = \sum -(y_i - f(x_i; w))^2 \text{ (2 marks)}$$

- (c) For gradient descent we aim for Maximum Likelihood Estimate –

$$w_{MCLE} = \arg \max_w \sum_i -(y_i - f(x_i; w))^2 \text{ (0.5 mark)}$$

$$= \arg \min_w \sum_i (y_i - f(x_i; w))^2 \text{ (0.5 mark)}$$

$$\frac{\partial \sum_i (y_i - f(x_i; w))^2}{\partial w^j} = \sum_i 2(y_i - f(x_i; w)) \frac{\partial (y_i - f(x_i; w))}{\partial w^j} \text{ (2 marks)}$$

$$= \sum_i -2(y_i - f(x_i; w)) \frac{\partial f(x_i; w)}{\partial w^j} \text{ (1 mark)}$$

Since $f(x) = w_0 + \sum_i w_i x_i$.

the gradient update rule will be

$$w^j \leftarrow w^j + \eta \sum_i (y_i - f(x_i; w)) x_i^j \text{ (2 marks)}$$

Here the subscripts denote the sample number and the superscript j denotes the j^{th} element of the vector x or w .

- (d)
- $y \sim \mathcal{N}(f(x; w), \sigma)$

From Bayes rule we know that

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

$$p(y|x)p(x) \propto p(y)$$

$$\Rightarrow \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{y-f(x)}{\sigma}\right)^2} p(x) \text{ (1 mark)}$$

$$\Rightarrow \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{y-f(x)}{\sigma}\right)^2} e^{-\lambda|x|} \text{ (1 mark)} \Rightarrow \ln \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{y-f(x)}{\sigma}\right)^2} e^{-\lambda|x|} \text{ (1 mark)}$$

$$\Rightarrow \ln e^{-\frac{1}{2}\left(\frac{y-f(x)}{\sigma}\right)^2} + \ln e^{-\lambda|x|} \text{ (1 mark)}$$

$$\Rightarrow -(y - f(x; W))^2 - \lambda|x|$$

Which is the expression for Lasso/L1 Regularisation. (1 mark)

II.2. (6 points) [END2018 Q1(c)]

The sum of squared error loss term for linear regression ($\mathbf{y} = \mathbf{X}\boldsymbol{\theta}$) given by the quadratic expression ($\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$), where each element of $\mathbf{y}$ and each row of $\mathbf{X}$ is one training sample pair. Starting with this loss, derive the closed form expression for the optimal parameters $\boldsymbol{\theta}^*$. Does ridge regression also have a closed form solution? If yes, what is it? If not, why not?

Solution: We have to minimize $(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})$ over all $\boldsymbol{\theta}$. So differentiating the

objective function with respect to the variables and equating it to zero, we get

$$\begin{aligned}
 & \frac{\partial}{\partial \boldsymbol{\theta}} \left((\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \right) = 0 \\
 \implies & \frac{\partial}{\partial \boldsymbol{\theta}} \left((\mathbf{y}^\top \mathbf{y} - 2\mathbf{y}^\top \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\boldsymbol{\theta}) \right) = 0 \\
 \implies & -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} = 0 \\
 \implies & 2\mathbf{X}^\top \mathbf{X}\boldsymbol{\theta} = 2\mathbf{X}^\top \mathbf{y} \\
 \implies & \boldsymbol{\theta} = \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{y}
 \end{aligned}$$

Yes, there exists a closed form solution for the ridge regression and has the form $\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$, where $\mathbf{I}$ is the identity matrix and λ is the regularization parameter.

II.3. (4 points) [END2018 Q1(b)]

What are the pros and cons of ridge regression and LASSO?

Solution:

- Ridge regression has a closed form solution, LASSO does not.
- Ridge regression provides stability in case of small sample, high-dimensional data. On the other hand the results from LASSO may not be very stable, i.e., small perturbations in the data may lead to significant changes in the coefficients.
- LASSO allows feature selection as it promotes sparsity. On the other hand, Ridge regression will lead to small values of the coefficients, but not zero.

II.4. (4 points) [MID2017 Q2(a) $\equiv$ END2019 Q5(a)]

What effect will adding a regularization term to a linear/logistic regression model have on the bias and variance of your model? Mention ‘increase’, ‘decrease’ or ‘no change’ along with a justification supporting your choice.

Solution: The bias increases (and variance decreases) if a regularization term is added to the objective of the linear/logistic regression model.

$$J(\boldsymbol{\theta}) = \mathcal{L}(\mathcal{D}, \boldsymbol{\theta}) + \lambda \mathcal{R}(\boldsymbol{\theta})$$

where, in case of linear regression the data loss term is $\mathcal{L}(\mathcal{D}, \boldsymbol{\theta}) = \sum_{i=0}^{N-1} (y_i - \boldsymbol{\theta}^\top \mathbf{x}_i)^2$ and the regularization term is $\mathcal{R}(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2$ (in case of a Gaussian prior/ ridge regression). Since adding a regularizer is equivalent to imposing a prior distribution on the parameters, it implies the solution will be *more consistent* with the prior parameter distribution and *less consistent* with the data.

Consider the extreme case when the regularization parameter is very large ($\lambda \gg 1$), the solution will practically ignore the data term, leading to a solution which is always the same, regardless of the data. Thus the train and test error rates will be similar, leading to a **low variance**, however, the *absolute* error will be much larger than the *true* error, implying a **high bias**. A similar argument can be made for logistic regression.

Alternative solution (END2019 Q5(a)): Yes, adding a regularizer will help. (1 mark)
This helps in lowering down the model complexity by imposing sparsity in the weights.

Consider for example, imposing LASSO and Ridge regression leads to reducing most of the weights / coefficients to zero or reducing their magnitude (closer to zero) respectively. This helps in reducing the variance by making the estimator simpler. (2 marks) There is an increase in the bias due to the additional information imposed. (1 mark)

II.5. (3 points) [MID2016 Q2]

A locally weighted linear regressor outputs the mean of k nearest neighbors of a test point. Given the training data as $\{x_0 = 0.1, x_1 = 0.3, x_2 = 0.6, x_3 = 0.8\}$, determine the output of the regressor for $k = 2$ over the interval $0 \leq x \leq 1$.

Solution: The regressor function is as follows:

$$f(x) = \begin{cases} 0.2 & 0 \leq x \leq 0.35 \\ 0.45 & 0.35 \leq x \leq 0.55 \\ 0.7 & 0.55 \leq x \leq 1 \end{cases} \quad (1)$$

II.6. (7 points) [END2017 Q6 $\equiv$ END2019 Q2(a)]

(7=2+1+1+2+1 points) Suppose you work for this to-become-great financial startup that gave you a lot of data for stock market prediction. Domain expertise and some quick verification of data led you to believe that the relationship between the input and the output variables are not linear. Unfortunately, you (or your more experienced colleagues) do not know the order of polynomial that will result in a good fit. However, you found that the data is smoothly varying. Given that you have limited computing resources (servers with large RAMs, but no GPUs) and are unable to train a deep neural network, your advisors have recommended using some variant of linear regression for quick prototyping. Succinctly answer the following questions for this scenario.

- What would be your modified loss function for the prediction problem?
- What would be the hyperparameters and how would you select them?
- How would you check if your model is correctly trained or not?
- Provide two different regularizers that you could use with these models? Also list one advantage each.
- Given the setup, what kind of an optimization technique would you chose? Provide a one line justification for your answer.

Hint: Smoothly varying data implies samples in a local neighborhood would have similar properties/structure. Do you know of any way this information could be leveraged?

Solution:

- Since there is a lot of data and it is smoothly varying, instead of linear regression, we can combine the idea of k nearest neighbor based weights and regression. The resulting technique is also known as locally weighted regression. The usual objective function is modified by incorporating the weights, which are determined based on the proximity to a query point. An example of the weighting function could be the Gaussian kernel function.

Let $\mathbf{x}_i^{(j)}$ is the i^{th} element of the j^{th} sample, where $i = 1, \dots, D$ and $j = 1, \dots, N$. Given a query point $\mathbf{x}_q$, the weights for each training point (or k -nearest neighbors) is

a function of its distance from the query. Nearer points contribute more and farther points contribute less. The error function (for each query point $\mathbf{x}_q$) can now be defined in one of the following ways:

•

$$E(\mathbf{x}_q) = \sum_{\mathbf{x} \in \mathcal{N}(\mathbf{x}_q)} \left(f(\mathbf{x}) - \hat{f}(\mathbf{x}) \right)^2$$

where $\mathcal{N}(\mathbf{x}_q)$ is the neighborhood of $\mathbf{x}_q$ containing its k nearest neighbors.

•

$$E(\mathbf{x}_q) = \sum_{i=1}^N K(d(\mathbf{x}, \mathbf{x}_q)) \cdot \left(f(\mathbf{x}) - \hat{f}(\mathbf{x}) \right)^2$$

where $K(d(\mathbf{x}, \mathbf{x}_q))$ could be computed using the Gaussian kernel function $k(\cdot)$.

•

$$E(\mathbf{x}_q) = \sum_{\mathbf{x} \in \mathcal{N}(\mathbf{x}_q)} K(d(\mathbf{x}, \mathbf{x}_q)) \cdot \left(f(\mathbf{x}) - \hat{f}(\mathbf{x}) \right)^2$$

It is easy to see that this *nonparametric* approach will be computationally intensive (with each query requiring its own fitting, i.e., parameter estimation) and also has a large memory footprint because you would need to store the entire training set.

b) For k -nearest neighbor based weights, the hyperparameter will be k . For Gaussian kernel based weights, the hyperparameter will be the bandwidth parameter σ .

c) Ensure the following:

- training error has stabilized (learning algorithm converged).
- verify that testing and training error are similar across various train/test splits
- Based on domain knowledge, if we have a reasonable guess of the magnitude of fitting errors, we can ensure that the model is not underfitting.

d) We could use ℓ_1 and ℓ_2 regularizers, with the former aiding variable selection by promoting sparsity of the parameters, and the latter helps in avoiding overfitting.

e) For locally weighted regression, a batch gradient descent type approach would be more effective.

Alternative solution (END2019 Q2(a)): Since the polynomial order is unknown, we will have to use cross-validation for model selection. Split the data into train, val and test sets and perform linear regression with increasing orders of transformations $\{x, x^2, x^3, \dots\}$. Use validation error (MSE) to establish a good fit. Alternatively, an appropriate kernel could be used, but again, cross-validation should be used to establish the goodness of fit for the linear regression problem.

II.7. (3 points) [MID2016 Q6]

(Extra credit) Let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be i.i.d. data from a uniform distribution over the disc of radius θ in R^2 , i.e., $\mathbf{x}_i \in R^2$ and

$$p(\mathbf{x}; \theta) = \begin{cases} \frac{1}{\pi\theta^2}, & \|\mathbf{x}\| \leq \theta \\ 0 & \text{otherwise} \end{cases}$$

where $\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2}$. What is the maximum likelihood estimate of θ .

Solution: The following log likelihood function will be finite only for $\theta \geq \|\mathbf{x}_i\|$, $i = 1, \dots, n$. It will be $-\infty$ otherwise.

$$\begin{aligned}\mathcal{L}(\theta) &= \ln \prod_{i=1}^n \frac{1}{\pi \theta^2} \\ &= \sum_{i=1}^n \ln \frac{1}{\pi \theta^2} \\ &= (-2 \ln \theta - \ln \pi) \sum_{i=1}^n 1 \\ &= -2n \ln \theta - n \ln \pi\end{aligned}$$

Since we need to maximize the log-likelihood, we have the following problem:

$$\begin{aligned}\min_{\theta} \quad & \ln \theta \\ \text{s.t.} \quad & \theta \geq \|\mathbf{x}_i\| \quad i = 1, \dots, n\end{aligned}\tag{1}$$

We can write the Lagrangian using all the n constraints, however, it will not give us the closed form solution. But by inspection of the problem above, it is straightforward to see that the maximum likelihood estimate (MLE) is

$$\hat{\theta}_{MLE} = \max_{i \in \{1, \dots, n\}} \|\mathbf{x}_i\|$$

II.8. (10 points) [MID2018 Q5]

{Extra Credit}: There is a heavy traffic jam at Connaught Place. There is bumper-to-bumper traffic and cars all around the outermost circle are at a standstill. Somehow Google leaks the GPS locations of all these cars. The typical noise in phone based GPS measurements would be around 10m. Google wants you to estimate the circumference of the outermost circle of Connaught Place (Trump asked them not to use other resources, but ask IIITD students to solve it). Since you have only done linear regression, you need to apply linear regression for this problem. First of all, can you apply *linear* regression? What would you need to do? Explain the entire training and validation pipeline.

Solution: Since we are given the latitude(X) and longitude(Y) of all the vehicles in the traffic. we can mean normalize both X and Y . Considering $(0,0)$ as the origin, we can transform the feature space by including X^2 and Y^2 . So, Now we have:

$$aX^2 + bY^2 + cX + dY = R^2$$

where R is the distance between the point and the center of CP. Now, we can fit linear regression to solve it. However, we can solve this problem in multiple other ways.

Part III Logistic Regression and Classification Losses

III.1. (2 points) [MID2021 Q1(a)]

What is the loss function used to train a logistic regression model? Define the probabilistic model used, and write the mathematical expression for the loss (log-likelihood).

Solution: The loss function used in Logistic Regression is log loss also known as Cross Entropy loss.

The probabilistic model used in Logistic Regression is Bernoulli (for two class classification):

$$Pr(Y = y|x; p) = p^y(1 - p)^{1-y}$$

The mathematical expression for loss (log likelihood) :

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i \log(p(x_i, \theta)) + (1 - y_i) \log(1 - p(x_i, \theta)))$$

where $p(x_i, \theta)$ is the predicted probability for the i^{th} sample. On simplification, we get,

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i \theta^T x_i - \log(1 + e^{\theta^T x_i}))$$

III.2. (2 points) [MID2021 Q1(b) $\equiv$ END2019 Q2(c)]

Write the regularized loss function for logistic regression with a Gaussian prior and a Laplacian prior (doubly exponential) on the weights. Which regularization function does each correspond to, and will the Laplacian prior have the same sparsity-inducing effect that it has in LASSO? Explain your answer.

Solution: For Gaussian Prior: $\mathcal{N}(0, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$;

Solving this with above defined loss function $L(\theta)$ will give:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i \theta^T x_i - \log(1 + e^{\theta^T x_i})) + \frac{\lambda}{2n} \sum_{j=1}^d \theta_j^2$$

where $\lambda = \frac{1}{\sigma^2}$

For Laplacian Prior with 0 mean: $\frac{1}{2b} e^{-\frac{|x|}{b}}$

Solving this with above defined loss function $L(\theta)$ will give:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i \theta^T x_i - \log(1 + e^{\theta^T x_i})) + \frac{\lambda}{n} \sum_{j=1}^d |\theta_j|$$

where $\sigma^2 = 2b^2$ and $\lambda = \frac{\sqrt{2}}{\sigma}$

n = Number of Samples, d = Number of Features

(Note: The constants in both the equations are ignored while expanding)

Alternative solution (END2019 Q2(c) – and the answer to the sparsity question): ℓ_1 -norm is used for regularization, and it assumes a Laplacian or doubly exponential distribution on the parameters $p(\mathbf{w}) = \exp(-\lambda |\sum_i w_i|)$. Derivation of the loss is through the standard process of MAP estimation. The sparsity will be retained as the loss function is still convex and the priors on the parameters will act in the same manner as in LASSO.

III.3. (2 points) [MID2021 Q1(c) $\equiv$ END2019 Q4(c)]

How would you extend or modify the binary classification loss to handle three (or more) different classes? What kind of activation would you have in the output layer of a neural network designed for multi-class classification?

Solution: In binary classification, where the number of classes ‘n’ equals 2, log loss or cross-entropy can be calculated as:

$$-(y \log(p) + (1 - y) \log(1 - p))$$

If number of classes, $n > 2$ (i.e., multiclass classification), we calculate a separate loss for each class label per observation and sum the result.

So, for multi-class classification, log loss can be calculated as:

$$-\sum_{i=1}^n y_{o,c} \log(p_{o,c})$$

where n = Number of classes

For 3 classes (for example, dog, cat, fish), $n = 3$.

y = Binary indicator (0 or 1) if class label c is the correct classification for observation o.

p = Predicted probability observation o is of class c.

Hence, for three different classes, log loss can be calculated as:

$$-\sum_{i=1}^3 y_{o,c} \log(p_{o,c})$$

Intuitively, total loss for 3 classes ‘dog’, ‘cat’ and ‘fish’ is as follows:

$$TotalLoss = CrossEntropy_{dog} + CrossEntropy_{cat} + CrossEntropy_{fish}$$

Alternative solution (END2019 Q4(c) – softmax output activation and the derivation): The logistic function can be used for the classification between two target classes $t = 1$ and $t = 0$, this can be generalized to output a multiclass categorical probability distribution by the softmax function.

(1 marks)

The softmax function S takes C –dimensional vector x as input and outputs a C –dimensional vector y of real values between 0 and 1. Mathematically softmax function is defined as below:

$$y_c = S(\mathbf{x})_c = \frac{e^{x_c}}{\sum_{d=1}^C e^{z_d}} \quad \text{for } c \in \{1 \dots C\} \quad (1)$$

$$\begin{bmatrix} p(t=1|\mathbf{x}) \\ \vdots \\ p(t=C|\mathbf{x}) \end{bmatrix} = \begin{bmatrix} S(\mathbf{x})_1 \\ \vdots \\ S(\mathbf{x})_C \end{bmatrix} = \frac{1}{\sum_{d=1}^C e^{z_d}} \begin{bmatrix} e^{z_1} \\ \vdots \\ e^{z_C} \end{bmatrix} \quad (2)$$

Where $P(t=x|\mathbf{x})$ is the probability that the class is c given the input x .

(2 marks)

Cross-entropy loss for softmax function:

The likelihood $\ell(\theta|\mathbf{t}, \mathbf{x})$ can be written as a conditional distribution $P(\mathbf{t}, \mathbf{x}|\theta)$ can be written as:

$$P(\mathbf{t}|\mathbf{x}, \theta) = P(\mathbf{t}|\mathbf{x}, \theta)P(\mathbf{x}|\theta) \quad (3)$$

For a fix θ and since each t_i is dependent on the $\mathbf{x}$ and only 1 class can be activated in the $\mathbf{t}$ we can write.

$$P(\mathbf{t}|\mathbf{z}) = \prod_{i=c}^C P(t_c|\mathbf{x})^{t_c} = \prod_{i=c}^C S(\mathbf{x})^{t_c} = \prod_{i=c}^C (y_c)^{t_c} \quad (4)$$

$$-\log \ell(\theta|\mathbf{t}, \mathbf{x}) = \xi(\mathbf{t}, \mathbf{x}) = -\log \prod_{i=c}^C (y_c)^{t_c} = -\sum_{i=c}^C t_c \log(y_c). \quad (5)$$

Which is the cross entropy function ξ . You may verify that for two class system output $t_2 = 1 - t_1$. The error function become $\xi(\mathbf{t}, \mathbf{y}) = -t_c \log(y_c) - (1 - t_c) \log(1 - y_c)$. The cross-entropy error function over a batch of multiple samples of size n can be calculated as:

$$\xi(T, Y) = \sum (\mathbf{t}_i, \mathbf{y}_i)_{i=1}^n = -\sum_{i=1}^n \sum_{i=c}^C t_{ic} \log(y_{ic}) \quad (6)$$

Where t_{ic} is 1 if and only if sample i belongs to class c and y_{ic} is the output probability that sample i belongs to class c .

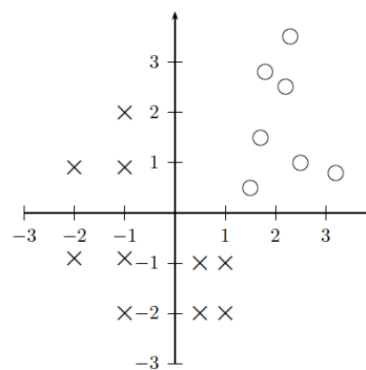
(2 marks)

III.4. (10 points) [END2018 Q1(a)]

Consider the regularized logistic regression loss function for the data shown in the figure

$$\max \sum_{i=1}^N \log \left(P(y^{(i)} | x_1^{(i)}, x_2^{(i)}) \right) - (\lambda_0 w_0^2 + \lambda_1 w_1^2 + \lambda_2 w_2^2)$$

where i indexes the i^{th} data point and $P(y|x_1, x_2) = \frac{1}{e^{-(w_0 + w_1 x_1 + w_2 x_2)}} + 1$. Describe what would be the learned weights (w_0, w_1, w_2) be in the following four scenarios. Instead of giving exact values of w_i , reason about what values would they take in the context of λ_i .



- (i) $\lambda_0 = \lambda_1 = \lambda_2 = 0$ (ii) $\lambda_0 = \infty; \lambda_1 = \lambda_2 = 0$
 (iii) $\lambda_0 = \lambda_1 = 0; \lambda_2 = \infty$ (iv) $\lambda_0 = \lambda_2 = 0; \lambda_1 = \infty$
 (v) $\lambda_0 = \lambda_1 = \lambda_2 = 1/3$

Solution: Assuming that w_1 and w_2 correspond to the horizontal and vertical axes respectively.

- (i) Since the data is linearly separable, and $\forall i, \lambda_i = 0$, we will have the boundary that logistic regression will generate. All $w_i \neq 0$, with the linear boundary passing between the two classes.

- (ii) Since $\lambda_0 = \infty$, w_0 cannot have a non-zero value or the loss function would go to infinity. Therefore the resulting boundary will have $w_0 = 0$, implying the boundary will pass through the origin.
- (iii) With $\lambda_2 = 0$, we have $w_2 = 0$, therefore the boundary will be a vertical line with an offset w_0 .
- (iv) With $\lambda_1 = 0$, we have $w_1 = 0$, therefore the boundary will be a horizontal line with an offset w_0 .
- (v) Since all λ_i are the same, the resulting boundary is expected to separate the data as in (i), but will be closer to the origin, as the regularizer will add to the bias.

III.5. (5 points) [MID2016 Q5]

Consider the following training set of four labeled data:

x_1	x_2	y
1	2	1
3	2	1
2	2	0
0	3	0

The logistic regression model is: $p(y = 1 | \mathbf{x} = [x_1 \ x_2]^\top) = \frac{1}{(1 + e^{-\mathbf{x}^\top \boldsymbol{\theta}})}$, where $\mathbf{x}^\top \boldsymbol{\theta} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$. Estimate the parameters θ_0 , θ_1 , and θ_2 .

Solution: Given, number of observation $N = 4$

Logistic model is given by

$$\begin{aligned} p(Y = 1 | \underline{x}) &= \frac{1}{1 + e^{-\underline{x}^\top \underline{\theta}}} \\ &= \frac{1}{1 + e^{-z}}, \quad z = \underline{x}^\top \underline{\theta} \end{aligned} \tag{1}$$

$$p(Y = 0 | \underline{x}) = 1 - p(Y = 1 | \underline{x}) = \frac{1}{1 + e^z}$$

Use maximum likelihood estimates –

$$\begin{aligned} \hat{\theta} &= \max_{\underline{\theta}} \left(\prod_{i=1}^4 p(Y_i = y_i | \underline{x}_i, \underline{\theta}) \right) \\ &= \max_{\underline{\theta}} \left\{ \prod_{i=1}^4 p(Y_i = 1 | \underline{x}_i, \underline{\theta})^{Y_i} p(Y_i = 0 | \underline{x}_i, \underline{\theta})^{(1-Y_i)} \right\} \end{aligned}$$

Log Likelihood function $\ln(l(\theta)) = L(\theta)$

$$\hat{\theta} = \max_{\underline{\theta}} (L(\theta))$$

$$= \max_{\underline{\theta}} \left\{ \sum_{i=1}^4 Y_i \ln[p(Y_i = 1 | \underline{x}_i, \underline{\theta})] + \sum_{i=1}^4 (1 - Y_i) \ln[p(Y_i = 0 | \underline{x}_i, \underline{\theta})] \right\}$$

$$L'(\underline{\theta}) = \frac{\partial L(\underline{\theta})}{\partial \underline{\theta}} = \sum_{i=1}^4 \underline{x}_i \{Y_i - p(Y_i = 1|\underline{x}_i, \underline{\theta})\} = X^T(\underline{y} - \underline{p})$$

where

$$X^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 3 & 2 \\ 1 & 2 & 2 \\ 1 & 0 & 3 \end{bmatrix}; \quad y = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$L''(\underline{\theta}) = \frac{\partial^2 L(\underline{\theta})}{\partial \underline{\theta} \partial \underline{\theta}^T}$$

Using Newton's method –

$$\underline{\theta}^{new} = \underline{\theta} - \frac{L'(\underline{\theta})}{L''(\underline{\theta})}$$

$$\rightarrow \underline{\theta}^{new} = (X^T W X)^{-1} X^T W \underline{Z}, \quad \underline{Z} = X \underline{\theta} + W^{-1}(\underline{y} - \underline{p})$$

Initialization, Let $\underline{\theta}^o = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

$$p_1^0 = \frac{1}{1 + e^{-\underline{\theta}^T \underline{x}_i}} = \frac{1}{1 + 1} = \frac{1}{2}$$

$$p_2^o = \frac{1}{2}, \quad p_3^o = \frac{1}{2}, \quad p_4^o = \frac{1}{2}$$

$$\therefore \underline{p}^o = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \end{bmatrix}$$

$$W_{ii} = p_i(1 - p_i)$$

$$\underline{z}^o = X \underline{\theta}^o + W^{-1}(\underline{y} - \underline{p}) = \begin{bmatrix} 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}$$

$$\underline{\theta}^{new} = (X^T W^o X)^{-1} X^T W^o \underline{z}^o$$

Substitute for $X, W^o, \underline{z}^o$ and repeat.

III.6. (5 points) [END2019 Q2(b)]

You have points along the surface of two concentric spheres in 3D, each belonging to a different class. How would you apply logistic regression to classify these samples?

Solution: Use a transformation to map to radius (or alternatively use a second order transform, i.e., $\{x, y, z\} \Rightarrow \{x^2, y^2, z^2\}$, or a mapping to spherical coordinates). Since it is a sphere, cross terms are not necessary. A kernel would be an overkill as you know the structure of the data that you need to classify.

Part IV Naive Bayes and Generative Classifiers

IV.1. (5 points) [MID2018 Q4(a)]

Explain why the most probable label for the training sample $(x_1, x_2, \dots, x_m)$ is the label l that maximizes the following:

$$P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m | L = l)P(L = l)$$

Solution: The most probable label for the training sample $X = (x_1, x_2, \dots, x_m)$ is one that maximizes $P(L = l | X = (x_1, x_2, \dots, x_m))$. Now,

$$\begin{aligned} P(L = l | X = (x_1, x_2, \dots, x_m)) &= P(L = l | X_1 = x_1, X_2 = x_2, \dots, X_m = x_m) & (1) \\ &= \frac{P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m | L = l)P(L = l)}{P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m)} & (\text{Bayes rule}) \end{aligned} \quad (2)$$

Now, since we're interested in maximizing $P(L = l | X = (x_1, x_2, \dots, x_m))$ over l , $P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m)$ remains constant over all l , thus doesn't affect argmax (although affects the absolute value of max). Thus, the l maximizing $P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m | L = l)P(L = l)$ also maximizes $\frac{P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m | L = l)P(L = l)}{P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m)}$. Hence, the label l that maximizes $P(X_1 = x_1, X_2 = x_2, \dots, X_m = x_m | L = l)P(L = l)$ is the most probable label.

IV.2. (5 points) [MID2017 Q3(b) $\equiv$ MID2018 Q4(b)]

To build a spam/no-spam classifier that uses text frequency features and uses a small training set, one will need to consider the scenario where some features have rare values that were *never* observed in the training data (e.g., the word 'Thiruvananthapuram' does not appear in training for an email classification problem). Assume that a word is never observed in the training set, but is present in a test email sample. Does such a situation pose a challenge for the Naive Bayes Classifier? If yes, how would you fix this issue? If not, then what strategy helps prevent this issue in NB?

Solution: If the NB is trained with Laplace Smoothing, then this issue does not pose a challenge for NB.

Alternative solution (MID2018 Q4(b) – the mechanism in full): In context of Naive Bayes classifier, smoothing corresponds to adding “virtual counts” for the purpose of calculating probabilities. Normally,

$$P(X_i = u | Y = y) = \frac{\text{Count}(X_i = u \wedge Y = y)}{\text{Count}(Y = y)} \quad (1)$$

In smoothing, this changes to:

$$P(X_i = u | Y = y) = \frac{\text{Count}(X_i = u \wedge Y = y) + 1}{\text{Count}(Y = y) + \text{number_of_classes}} \quad (2)$$

Smoothing is required in Naive Bayes training, because in the training set, we may not observe a particular feature in a particular class (we clearly would not necessarily see all possible features in the training set since it's not exhaustive), Naive Bayes without smoothing will see that $P(X_i = u|Y = y) = 0$ in the training data, thus will assign 0 probability to class y for any sample that has $X_i = u$. This is worse when a particular feature value hasn't been seen at all before (say, with integer data). If the training data doesn't have $X_i = u$ in any sample, Naive Bayes without smoothing assigns 0 probability to each class, which doesn't give us a good decision. Instead, it would be good if the classifier looked at other feature values in such a case. Smoothing helps ensure exactly this.

IV.3. (6 points) [END2017 Q4]

Consider the Naive Bayes classifier trained on the following dataset:

x_1	x_2	x_3	y
1	1	1	1
1	0	1	1
1	1	0	1
0	0	1	0
0	0	0	0

What is the prediction on y for the input sample's feature $x_1 = 1, x_2 = 0, x_3 = 0$?

Solution:

[Handwritten solution, reproduced from the scanned source pages.]

Naive Bayes assumes all features to be statistically independent

Now $P(c_1) = \frac{2}{5}$ $P(c_2) = \frac{3}{5}$, where $c_1 = 0$ class-1 $c_2 = 1$ class-2

Now $P(x_1/c_1) = 0/2 = 0$ $P(x_2/c_1) = 0/2 = 0$ $P(x_3/c_1) = 1/2$
 $P(x_1/c_2) = 2/3 = 1$ $P(x_2/c_2) = 2/3$ $P(x_3/c_2) = 2/3$

Rule: say c_1 if $P(c_1/x) > P(c_2/x)$

$P(c_1/x) = \frac{P(x/c_1)P(c_1)}{P(x)} = \frac{P(x_1/c_1)P(x_2/c_1)P(x_3/c_1)P(c_1)}{P(x)} = \frac{0}{P(x)}$

$P(c_2/x) = \frac{P(x/c_2)P(c_2)}{P(x)} = \frac{P(x_1/c_2)P(x_2/c_2)P(x_3/c_2)P(c_2)}{P(x)} = \frac{1 \times \frac{2}{3} \times \frac{2}{3} \times \frac{3}{5}}{P(x)}$

Since $P(c_2/x) > P(c_1/x)$
 $\therefore$ Say class c_2
Hence, $y = 1$

IV.4. (8 points) [END2017 Q3]

(8=2+3+3 points) We are given a two class dataset with class labels $y \in \{0, 1\}$ described by a single feature x . The dataset has equal number of samples of each class. Say, we decide to build a binary Gaussian Naive Bayes (GNB) Classifier.

(a) (2 points) Write the expression of the decision rule.

Solution: The decision rule of GNB classifier is given by:

Decide class c_1 if $P(c_1/x) > P(c_2/x)$, else decide c_2 .

- (b) (3 points) We are given that the feature of the two classes have unequal means ($\mu_1 = 0$ and $\mu_2 = 2$) but equal variances ($\sigma_1^2 = 1$). Is the decision boundary linear? What is the expression of the decision boundary? Justify your answer. Draw the graph showing the distribution under the two classes along with the decision boundary.

Solution:

[Handwritten solution, reproduced from the scanned source pages.]

$$\begin{aligned}
 P(C_1/x) &= \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-0)^2}{2}} \\
 P(C_2/x) &= \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-2)^2}{2}}
 \end{aligned}$$

Say c_1 if $\frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} > \frac{1}{\sqrt{2\pi}} e^{-\frac{(x-2)^2}{2}}$

if $-\frac{x^2}{2} > -\frac{x^2 + 4x - 4}{2}$

Say c_1 , if $x < 1$ $\Rightarrow$ decision boundary is linear
 else say c_2

- (c) (3 points) We are given that the feature of the two classes have equal means ($\mu_1 = \mu_2 = 0$) but unequal variances ($\sigma_1^2 = 0.25$ and $\sigma_2^2 = 1$). What is the expression of decision boundary? Justify your answer. Draw the graph showing the distribution under the two classes along with the decision boundary.

Solution:

[Handwritten solution, reproduced from the scanned source pages.]

$$\begin{aligned}
 P(C_1/x) &= \frac{1}{\sqrt{2\pi\sigma_1^2}} e^{-\frac{x^2}{2\sigma_1^2}} \\
 P(C_2/x) &= \frac{1}{\sqrt{2\pi\sigma_2^2}} e^{-\frac{x^2}{2\sigma_2^2}}
 \end{aligned}$$

Say c_1 if $\frac{1}{\sigma_1\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma_1^2}} > \frac{1}{\sigma_2\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma_2^2}}$

$\ln \frac{\sigma_2}{\sigma_1} - \frac{x^2}{2\sigma_1^2} + \frac{x^2}{2\sigma_2^2} > 0$

$2 \ln \frac{\sigma_2}{\sigma_1} - x^2 \frac{(\sigma_2^2 - \sigma_1^2)}{\sigma_1^2 \sigma_2^2} > 0$

$x = \pm \sqrt{\frac{2 \ln \frac{\sigma_2}{\sigma_1}}{\frac{\sigma_2^2 - \sigma_1^2}{\sigma_1^2 \sigma_2^2}}}$

The handwritten working runs off the bottom of the scanned page in the source PDF; nothing further is recoverable.

IV.5. (2 points) [MID2017 Q3(a)]

In online learning, we can update the decision boundary of a classifier based on new data without reprocessing the old data. Now for a new data point that is an *outlier*, which of the following classifiers are likely to be affected more severely? Naive Bayes (NB), Logistic Regression (LR), Support Vector Machine (SVM)? {**Note:** An outlier is a point with lies in a low-probability region, typically farther away from most points of that class, e.g., for a gaussian random variable $\mathcal{N}(\mu, \sigma^2)$, an outlier would be a point that lies beyond the $\mu \pm 3\sigma$ range.}

Solution: Since NB and LR both rely on the estimating the density (or conditional density) of the points, an outlier is likely to influence them more severely. On the other hand, SVMs only rely on the support vectors to define the decision boundary. Moreover, the regularization helps in ignoring outliers in case of SVMs.

IV.6. (2 points) [MID2017 Q3(c)]

Now to build a classifier on high-dimensional features using small training data, one will need to consider the scenario where many features are just irrelevant noise. To train a generalizable classifier, would you want to use NB or LR? How will you augment the original formulation of the classifier under a Bayesian or regularization setting, i.e., which regularizer or prior will you incorporate in the likelihood term?

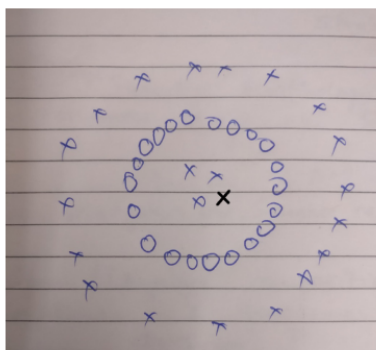
Solution: This noisy high-dimensional data requires us to perform feature selection, therefore it would make sense to use LR with an L_1 regularizer that promotes sparsity on the parameters. The parameters which turn out to be zero indicate the features that should be ignored. It is hard to incorporate such a feature in NB.

Part V Nearest Neighbours and Instance-Based Methods

V.1. (2 points) [MID2017 Q2(b)]

What effect will increasing k in k -nearest neighbor models have on the bias and variance of your model? Mention ‘increase’, ‘decrease’ or ‘no change’ along with a justification supporting your choice.

Solution: The bias increases (and variance decreases) as we increase the value of k in k -nearest neighbor classification. In k -nearest neighbor classification, the class label assignment is performed by (weighted) voting over k neighbors. As k increases, more distant neighbors will contribute to the label of the test sample, thus increasing the bias. Consider the example shown in the figure below, where the test sample is the black ‘x’ and the training examples are in blue ink. As the value of k is increased, all samples (train and test) closer to the center will get classified as ‘o’, which implies that the bias will be higher. However, since the train and test errors are similar, the variance reduces as k increases.



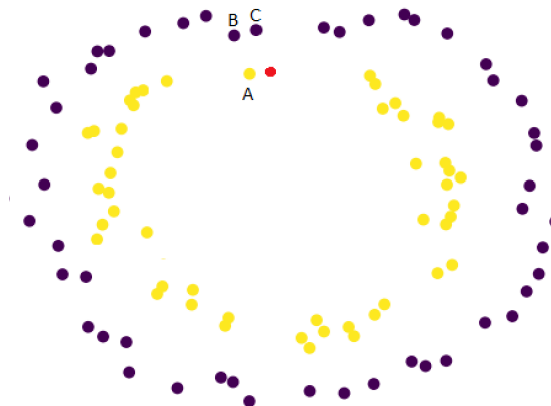
V.2. (6 points) [MID2018 Q4(c)]

True or False ?. Justify your answer.

- (i) In a nearest neighbor classifier, Euclidean distance and squared Euclidean distance are equivalent.
- (ii) The 3-nearest neighbor classifier is always more accurate than the 2-nearest neighbor classifier.
- (iii) With sufficient training data, the error of a nearest neighbor classifier always goes down to zero.

Solution:

- (i) True. Nearest neighbor example only compares distances to see which points are closer, and which are farther. It doesn't take into account the magnitude by which a point is closer or farther. Since distances are always non-negatives, $x > y \iff x^2 > y^2$ (since $x, y \geq 0$)
- (ii) False. The 3-nearest neighbor classifier is not always more accurate than the 2-nearest neighbor classifier. As an example, look at the following figure: When classifying the red point (test sample), we can clearly see that the point belongs to the yellow class. The 2-nearest neighbor classifier finds A and C as nearest neighbors, and thus classifies the point as yellow (tie breaking by distance). But, the 3-nearest neighbor classifier finds A,B,C as nearest neighbors and thus, classifies it into purple class.



The figure this part refers to. In the source PDF it is printed on the last page, physically above the Connaught Place question, but the A/B/C labels and the red test point tie it to this part.

- (iii) False. Since the k-nearest neighbor classifier is distance based, when data has distributions with some overlap across classes, k-NN may still make classification mistakes.

V.3. (2 points) [MID2018 Q4(d)]

Give scenarios where you would prefer to use k-nearest neighbors instead of Support Vector Machines.

Solution: Low-dimensional feature spaces, where classes have multi-modal distributions and non-linear boundaries.

V.4. (2 points) [MID2018 Q4(e)]

k-nearest neighbors is a nonparametric classifier because you need to retain all the training samples in order for it to work well. If I say SVMs are not nonparametric, would I be correct or wrong? Justify your answer.

Solution: Linear SVMs are parametric classifiers, since once the parameters $\mathbf{w}$ and $\mathbf{b}$ are identified, the entire training data can be discarded. Given the dimensionality of the data, the number of parameters is fixed. On the other hand, the number of parameters (no. of support vectors) required to define the decision rule depends on the number of training data. If you change the training set, the resulting number of support vectors may change, implying that the number of parameters is not fixed. Therefore kernelized SVMs are non-parametric.

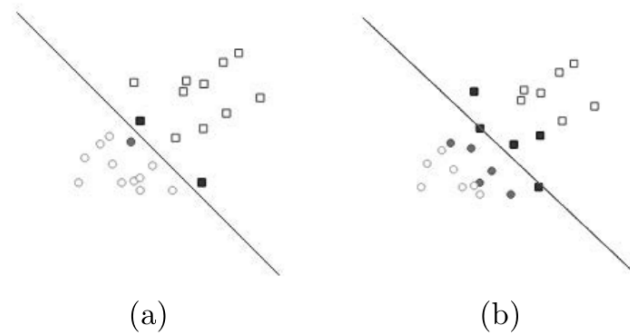
Part VI Support Vector Machines and Kernels

VI.1. (3 points) [MID2017 Q4 $\equiv$ QUIZ3 Q2]

- (a) Write down the soft-margin primal formulation of SVM. Let ‘C’ denote the regularization parameter that weights the classification loss term (not $w^\top w$). In the figure below, the solid lines denote the decision boundary, the *solid* squares (circles) indicate the support vectors and the squares (circles) denote the points in the input space. The two images are a result of using different kernel and different regularization parameters. Which of the following configurations of SVM would describe the images best? List all that apply to the figures (a) and (b), with one-line justification.
- soft-margin SVM with small C
 - soft-margin SVM with large C
 - hard-margin SVM with polynomial kernel, i.e., $k(x, y) = (x^\top y)^p$, $p > 1$
 - hard-margin SVM with radial basis function kernel, i.e., $k(x, y) = \exp(-\gamma \|x - y\|_2^2)$, $\gamma > 0$
 - none of the above.
- (b) The primal formulation for the hard-margin SVM is convex. Prove that the soft-margin objective is still convex.

Solution: (a)

$$\begin{aligned}
 \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^M \xi^{(i)} \\
 \text{s.t.} \quad & y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi^{(i)}, \quad i = 1, \dots, M \\
 & \xi^{(i)} \geq 0, \quad i = 1, \dots, M
 \end{aligned} \tag{1}$$



OR

$$\min_{\mathbf{w}, b} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^M \max\left(0, 1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b)\right)$$

Correction (as announced during the exam): The solid points were points that were on or within the margin.

- (a) In this case, we have a linear boundary and a small number of solid points (i.e., points violating the classification constraint), indicating that the margin is smaller. Such a condition implies that we have a linear SVM (owing to the linear boundary) with a large value of C (implying fewer margin violation). Therefore, this implies **case b**).
- (b) In this case, we again have a linear boundary but a larger number of solid points, which indicates there is more emphasis on having a larger boundary, i.e. the first term ($\|\mathbf{w}\|$) carries a larger weight (and consequently smaller value of C). Therefore, this implies **case a**).

Note that neither the radial basis function kernel, nor the polynomial kernel (with $p > 1$) will yield a linear boundary in the *input space*.

(b) [from QUIZ3 Q2] Soft-margin primal formulation for SVM is:

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i^2$$

s.t.

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 - \epsilon_i, i = 1, \dots, m$$

$$\epsilon_i \geq 0, i = 1, \dots, m$$

Standardising the inequality constraints,

$$1 - \epsilon_i - y^{(i)}(w^T x^{(i)} + b) \leq 0, i = 1, \dots, m$$

$$-\epsilon_i \leq 0, i = 1, \dots, m$$

We know that the primal formulation for the hard-margin is convex. The above formulation is still convex as the objective is a quadratic problem subject to the constraints being linear.

VI.2. (10 points) [QUIZ2 Q2 $\equiv$ END2018 Q2(a)(i); END2018 Q2(a)(ii,iii)]

- (a) Suppose you have identified the maximum margin separating hyperplane $\mathbf{w}^*$. Given points $\{\mathbf{x}_i^+\}$, $i = 1, 2, \dots, M$ and $\{\mathbf{x}_i^-\}$, $i = 1, 2, \dots, N$, that you want to classify, how

would you compute the optimal offset b^* ?

- (b) Consider the training data samples and the corresponding Lagrange multipliers learned from them, as given in the following table. Identify the support vectors.

i	x_i	y_i	α_i	i	x_i	y_i	α_i
1	(4,2.9)	1	0.414	6	(1.9,1.9)	-1	0
2	(4,4)	1	0	7	(3.5, 4)	1	0.018
3	(1,2.5)	-1	0	8	(0.5,1.5)	-1	0
4	(2.5,1)	-1	1.18	9	(2,2.1)	-1	0.414
5	(4.9,2.5)	1	0	10	(4.5,2.5)	1	0

- (c) Compute w and classify the point $(3,3)$.

Solution: (a) For two classes, we can write these equations:

$$y = w^*x + b^*$$

$$w^*x + b^* \geq 1$$

for data points of class 1

$$w^*x + b^* \leq -1$$

for data points of class -1

For any support vector, we would have,

$$\min_{i:y(i)=1} w^*x$$

$$\max_{i:y(i)=-1} w^*x$$

An optimal value of b would be one such that:

$$b^* = -\frac{\max_{i:y(i)=-1} w^*x + \min_{i:y(i)=1} w^*x}{2}$$

You may refer page 13 [here](#).

An alternate solution may show how support vectors are obtained from the dual form, and then proceed to use these values to calculate the distance to the optimal hyperplane, and calculate b^* appropriately.

Alternative solution (END2018 Q2(a)(i)): $w = \sum_{i:\alpha_i>0} \alpha_i y_i x_i$, and $b = y_k - w^\top x_k$, where x_k are all the support vectors.

- (b) The support vectors are the points with indices $i = \{1, 4, 7, 10\}$.

- (c) Applying $w^\top x + b$ and taking the sign for $x = [3, 3]^\top$ will yield the classification result.

VI.3. (5 points) [MID2016 Q4]

Consider the training dataset $\{(x_1, y_1) = (-2, -1), (x_2, y_2) = (-0.5, -1), (x_3, y_3) = (0.5, 1), (x_4, y_4) = (2, 1)\}$, where x_i are the data points and y_i are the corresponding label. If you chose to learn an SVM using this training data, how would you formulate the primal problem? What is the width of the margin in the input space? If we use a feature mapping $\phi(x_i) = [x_i \ x_i^2]^\top$ with this dataset, followed by learning a linear SVM in the new feature space, what would be the width of the margin? In general, if we use a mapping $\psi(x_i) = [x_i, x_i^2, \dots, x_i^n]$ for this data, how would the SVM margin change?

Solution: Since the data in input space is one dimensional, and appears to be linearly separable, there is only one $\mathbf{w}$ in the search space, which is a scalar, i.e., $w=1$. It is straightforward to verify that for $w=1$, the margin for this dataset is also 1.

For the mapping to $\mathbb{R}^2$ through the nonlinear function $\phi(x) = [x \ x^2]^\top$, again we can see that the Euclidean distance between the closest points does not change due to the symmetry about the second basis vector (vertical axis). Since these are only the two closest points from different classes, the separating hyperplane is their perpendicular bisector.

In general, the margin *will* change by increasing slightly. The third element of $\psi(x_2)$ and $\psi(x_3)$ will have opposite signs and will contribute to the additional Euclidean distance. It can be shown (how?) that the points $\psi(x_2)$ and $\psi(x_3)$ will still remain the closest points from opposite classes, regardless of what value n takes. Therefore, being the only two closest points of different classes, the optimal solution will be the perpendicular bisector of these points. The resulting margin width will be the Euclidean distance, which will change due to the elements with the odd exponents.

VI.4. (2 points) [MID2017 Q2(d)]

What effect will removing all the non-support vectors in SVMs have on the bias and variance of your model? Mention ‘increase’, ‘decrease’ or ‘no change’ along with a justification supporting your choice.

Solution: The SVM boundary *only* depends on the ‘support vectors’ and the remaining points do not affect the boundary. Therefore, there is no impact whatsoever by removing the non-support vector points. Hence, both the bias and the variance remain unchanged.

VI.5. (4 points) [END2017 Q2(a)]

What is the leave-one-out cross-validation error estimate for maximum margin separation in Figure-1 below?



Figure 1: Sample Space

Solution: Let us say that we have a total of N samples. Leave-one-out cross validation implies that we train on $N-1$ samples and test on the N^{th} sample (i.e., one sample that is left out). This process is repeated such that all N samples are tested once and the error is averaged out on all these N iterations.

From Figure-1, we note that the data is linearly separable and that there are 6 support vectors (3 for each class falling in a line at the center of the figure).

If we remove any of the sample other than these 6 support vectors and train the classifier, the same maximum margin SVM classifier would be learned and the test error on the sample chosen would be zero. If we remove any of the above support vectors, the decision boundary

would still remain the same and hence, the test error on this support vector chosen would be zero. In other words, the average error with LOO linear SVM classifier with the above sample space is zero.

VI.6. (8 points) [END2017 Q2(b)]

Given a standard linear binary SVM classifier.

$$\min_{\mathbf{w}} \frac{\mathbf{w}^\top \mathbf{w}}{2}, \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad i \in 1, 2, \dots, l. \quad (1)$$

Can we increase the physical margin width by defining

$$\min_{\mathbf{w}'} \frac{\mathbf{w}'^\top \mathbf{w}'}{2}, \quad \text{s.t.} \quad y_i(\mathbf{w}'^\top \mathbf{x}_i + b') \geq 2 \quad i \in 1, 2, \dots, l? \quad (2)$$

Justify your answer with mathematical reasoning.

Solution: Consider Figure-S1 below. The line L_1 corresponds to the hyperplane of positive class support vectors, line L_2 corresponds to the hyperplane of negative class support vectors, and the line L_0 corresponds to the decision boundary.

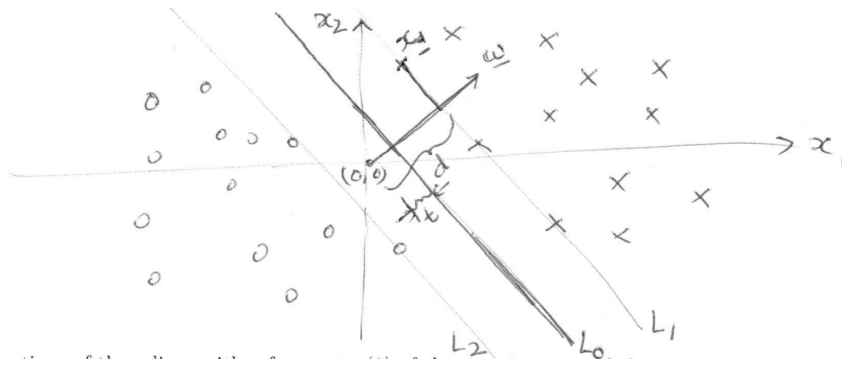


Figure S1 (hand-drawn in the scanned source).

The equations of these lines with reference to (1) of the question are as below:

$$L_1 : \mathbf{w}^\top \mathbf{x} + b = 1$$

$$L_0 : \mathbf{w}^\top \mathbf{x} + b = 0$$

$$L_2 : \mathbf{w}^\top \mathbf{x} + b = -1$$

Consider a sample $\mathbf{x}$ on line L_1 as shown in Figure-S1. The perpendicular distance of this point from the line L_0 corresponds to the margin from the decision boundary toward the positive class samples and is shown as $d_+ = d - t$. The value of d is given by the length of the projection of this point on the vector $\mathbf{w}$ as

$$d = \frac{\mathbf{w}^\top \mathbf{x}}{\|\mathbf{w}\|}$$

Any point $\mathbf{x}_0$ projected on line L_0 provides t , given by

$$t = \frac{\mathbf{w}^\top \mathbf{x}_0}{\|\mathbf{w}\|}$$

Since L_0 is defined as $L_0 : \mathbf{w}^\top \mathbf{x}_0 + b = 0$, $\mathbf{w}^\top \mathbf{x}_0 = -b$. Hence, t is given by $t = \frac{-b}{\|\mathbf{w}\|}$. Thus, $d_+ = d - t = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|}$. However, on line L_1 , any point will satisfy its equation given by $L_1 : \mathbf{w}^\top \mathbf{x} + b = 1$. Thus, $d_+ = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$.

Now, let us revisit the equation of line L_1 :

$$\mathbf{w}^\top \mathbf{x} + b = 1$$

On multiplying both sides by 2, we obtain:

$$2\mathbf{w}^\top \mathbf{x} + 2b = 2.$$

The above equation can be written as:

$$\mathbf{w}'^\top \mathbf{x} + b' = 2.$$

Computing the distances similar to the previous case, we would obtain $d_+ = \frac{\mathbf{w}'^\top \mathbf{x} + b'}{\|\mathbf{w}'\|} = \frac{2}{\|\mathbf{w}'\|} = \frac{1}{\|\mathbf{w}\|}$. Thus, it would bring us to the same margin.

This is intuitively correct. Since the training samples are at the same place, the maximum margin and the decision boundary should not change. Hence, changing the variables such that $\mathbf{w}'^\top \mathbf{x} + b' = M$ where $M > 1$ will not increase the margin because margin is normalized with respect to the norm of weight vector and also because the linear SVM formulation of equation (1) requires norm of this weight vector to be minimized.

VI.7. (8 points) [MID2017 Q5 $\equiv$ END2018 Q2(c)]

Given n training examples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, let elements of the $n \times n$ kernel matrix be $\mathbf{K}_{ij} = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$.

- (a) (3 points) Prove that the kernel matrix $\mathbf{K}$ is positive semidefinite. *Hint:* Recall that an $n \times n$ matrix $\mathbf{A}$ is positive semidefinite if and only if for all $\mathbf{y} \in \mathbb{R}^n$, we have $\mathbf{y}^\top \mathbf{A} \mathbf{y} \geq 0$.
- (b) (5 points) Mercer's theorem tells us that $K(x; z)$ is a valid kernel function if and only if for any finite set $\{x_1, \dots, x_m\}$, the square matrix $K \in \mathbb{R}^{m \times m}$ is symmetric and positive semidefinite, where the entries of the matrix are given by $K_{ij} = K(x_i; x_j)$. Recall that a positive definite matrix K satisfies $x^\top K x > 0$, $\forall x \in \mathbb{R}^m, x \neq 0$. K also has only positive eigenvalues, and every submatrix containing the diagonal elements of K , i.e., every submatrix of size $k \times k$, $k = 1, 2, \dots, m$ starting from the top left entry has a positive determinant.

Let K_1, K_2 be kernels over $\mathbb{R}^n \times \mathbb{R}^n$, let $a \in \mathbb{R}^+$ be a positive real number. For each of the functions K below, state whether it is necessarily a kernel. If you think it is, prove it; if you think it isn't, give a counter-example.

- (i) $K(x, z) = K_1(x, z) + K_2(x, z)$
- (ii) $K(x, z) = K_1(x, z) - K_2(x, z)$
- (iii) $K(x, z) = aK_1(x, z)$
- (iv) $K(x, z) = -aK_1(x, z)$
- (v) $K(x, z) = K_1(x, z)K_2(x, z)$

Solution:

- a) Let $\mathbf{L} = [\Phi(\mathbf{x}_1), \Phi(\mathbf{x}_2), \dots, \Phi(\mathbf{x}_n)]$, then we have $\mathbf{K} = \mathbf{L}^\top \mathbf{L}$. Now we can write $\mathbf{y}^\top \mathbf{K} \mathbf{y} = \mathbf{y}^\top \mathbf{L}^\top \mathbf{L} \mathbf{y} = (\mathbf{L} \mathbf{y})^\top \mathbf{L} \mathbf{y} = \hat{\mathbf{y}}^\top \hat{\mathbf{y}}$, which is a quadratic term and cannot take negative values. Therefore we can say that $\mathbf{y}^\top \mathbf{K} \mathbf{y} \geq 0$ and hence $\mathbf{K}$ is positive semidefinite.

- b) (i) $\mathbf{y}^\top \mathbf{K} \mathbf{y} = \mathbf{y}^\top \mathbf{K}_1 \mathbf{y} + \mathbf{y}^\top \mathbf{K}_2 \mathbf{y}$ is a sum of two nonnegative (positive) numbers and will always be nonnegative (positive), regardless of the value of $\mathbf{y}$.
- (ii) $\mathbf{y}^\top \mathbf{K} \mathbf{y} = \mathbf{y}^\top \mathbf{K}_1 \mathbf{y} - \mathbf{y}^\top \mathbf{K}_2 \mathbf{y}$ is difference of two nonnegative (positive) numbers and will only be nonnegative (positive) if the first term is larger. Therefore $\mathbf{K}$ cannot be a valid kernel.
- (iii) $\mathbf{y}^\top \mathbf{K} \mathbf{y} = \mathbf{y}^\top (a\mathbf{K}_1) \mathbf{y} = a(\mathbf{y}^\top \mathbf{K}_1 \mathbf{y})$ will always be nonnegative (positive) as long as $a > 0$, therefore $\mathbf{K}$ is a valid kernel.
- (iv) $\mathbf{y}^\top \mathbf{K} \mathbf{y} = \mathbf{y}^\top (-a\mathbf{K}_1) \mathbf{y} = -a(\mathbf{y}^\top \mathbf{K}_1 \mathbf{y})$ implies that if $a > 0$, then $\mathbf{K}$ is not a valid kernel (based on the argument in previous part).
- (v) This is slightly more complicated to prove.

$$\begin{aligned}
 \mathbf{K}_1(\mathbf{x}_1, \mathbf{x}_2) &= \Phi(\mathbf{x}_1)^\top \Phi(\mathbf{x}_2) \\
 \mathbf{K}_2(\mathbf{x}_1, \mathbf{x}_2) &= \Psi(\mathbf{x}_1)^\top \Psi(\mathbf{x}_2) \\
 \mathbf{K}(\mathbf{x}_1, \mathbf{x}_2) &= \mathbf{K}_1(\mathbf{x}_1, \mathbf{x}_2) + \mathbf{K}_2(\mathbf{x}_1, \mathbf{x}_2) \\
 &= (\Phi(\mathbf{x}_1)^\top \Phi(\mathbf{x}_2)) + (\Psi(\mathbf{x}_1)^\top \Psi(\mathbf{x}_2)) \\
 &= \left(\sum_{i=0}^{D_1} \phi_i(\mathbf{x}_1) \phi_i(\mathbf{x}_2) \right) + \left(\sum_{j=0}^{D_2} \psi_j(\mathbf{x}_1) \psi_j(\mathbf{x}_2) \right) \\
 &= \sum_{i,j \in D_1 \times D_2} \phi_i(\mathbf{x}_1) \phi_i(\mathbf{x}_2) \psi_j(\mathbf{x}_1) \psi_j(\mathbf{x}_2) \\
 &= \sum_{i,j \in D_1 \times D_2} (\phi_i(\mathbf{x}_1) \psi_j(\mathbf{x}_1)) (\phi_i(\mathbf{x}_2) \psi_j(\mathbf{x}_2))
 \end{aligned}$$

Now, if we define a new feature space $h_{i,j}(\mathbf{x}) = \phi_i(\mathbf{x}) \psi_j(\mathbf{x})$, then we get a new kernel matrix $\mathbf{K}(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{h}(\mathbf{x}_1)^\top \mathbf{h}(\mathbf{x}_2)$, which is written as an inner product of the new feature space defined by the mapping $\mathbf{h}(\cdot)$. This would hold for infinite dimensional D_1 and D_2 as well, as long as the feature spaces defined by $\phi(\cdot)$ and $\psi(\cdot)$ map the input vectors to a valid Hilbert space.

Alternative solution (END2018 Q2(c)): The necessary conditions for a valid kernel function is that it should generate a symmetric positive (semi) definite kernel matrix.

- (i) Given that $K_1(x, y)$ and $K_2(x, y)$ are valid kernel functions, let the corresponding kernel matrices be $\mathbf{K}_1$ and $\mathbf{K}_2$. Then, we know that $\forall \mathbf{x} \neq 0, \mathbf{x}^\top \mathbf{K}_1 \mathbf{x} \geq 0$, and $\mathbf{x}^\top \mathbf{K}_2 \mathbf{x} \geq 0$. Consider the matrix $\mathbf{K}$ generated from the proposed kernel $K(x, y) = K_1(x, y) + K_2(x, y)$ and the corresponding matrix $\mathbf{K} = \mathbf{K}_1 + \mathbf{K}_2$. Checking $\mathbf{x}^\top \mathbf{K} \mathbf{x}, \forall \mathbf{x} \neq 0$, we have $\mathbf{x}^\top \mathbf{K} \mathbf{x} = \mathbf{x}^\top (\mathbf{K}_1 + \mathbf{K}_2) \mathbf{x} = \mathbf{x}^\top \mathbf{K}_1 \mathbf{x} + \mathbf{x}^\top \mathbf{K}_2 \mathbf{x}$, which is a sum of two nonnegative numbers and is therefore nonnegative.
- (ii) A matrix cannot be symmetric positive (semi-) definite (PSD), if any of the diagonal elements is negative. Let $\mathbf{K}_1 = \mathbf{I}$ and $\mathbf{K}_2 = 2\mathbf{I}$, which are both PSD, but their difference is not.

VI.8. (4 points) [END2019 Q3(a)]

(2+2) The ξ^2 kernel is defined as $\kappa(x, y) = \sum_{i=1}^D \frac{x_i y_i}{x_i + y_i}$, where $x_i, y_i \geq 0$ and $\sum_{i=1}^D x_i = \sum_{i=1}^D y_i = 1$. Text data is often represented using feature vectors that are like histograms, frequently referred to as *bag of words*. Explain how would you use ξ^2 kernel to compute distances between two such feature points. Why is it justified to use this specific kernel?

{*Hint*: Recall that kernel functions represent inner products between vectors in some feature space. How can you use inner products to compute distances?}

Solution: (2 points) - Compute distance between 2 features.

$$K(x, y) = \sum_{i=1}^D \frac{x_i y_i}{x_i + y_i}$$

where $x_i, y_i \geq 0$ and $\sum_{i=1}^D x_i = \sum_{i=1}^D y_i = 1$

Kernel function represent dot products between vectors in some feature space. Euclidian distance between 2 points can be calculated as:

$$D(x, y) = \sum_{i=1}^D \|X_i - Y_i\|^2$$

$$D(x, y) = \sum (X_i - Y_i)(X_i - Y_i)^T$$

$$D(x, y) = \sum X_i^2 + Y_i^2 - 2X_i Y_i$$

$$D(x, y) = \sum X_i^T X_i + Y_i^T Y_i - 2X_i^T Y_i$$

$$D(x, y) = \sum_{i=1}^P K(X_i, X_i) + K(Y_i, Y_i) - 2K(X_i, Y_i)$$

(2 points) - Justification.

It is justified to use this specific kernel because $K(x, y) = \sum_{i=1}^D \frac{x_i y_i}{x_i + y_i}$ when X_i, Y_i are close, the value of $K(x_i, y_i)$ reduces and when the vectors are far away, the value of $K(x_i, y_i)$ increases.

The kernel matrix can be seen as the measure of similarity between 2 given vectors of the above kernel aligns with that intuition since the kernel values increase where x and y are similar and decrease when they are different.

VI.9. (8 points) [END2019 Q3(b)]

(2+6) What is complementary slackness with respect to a convex optimization problem? Does it apply to support vector Machine Learnings? Explain the implications of the complementary slackness condition by writing all the constraints on the primal and dual variables for the SVMs.

Solution: Suppose (α, β) are dual feasible. $\theta_D(\alpha, \beta) = nf(x^*) + \sum_{i=1}^n \alpha_i g(x_i^*) + \sum_{i=1}^P \beta_i h(x_i^*)$

(2 points)

Complementary slackness states that if strong duality holds, then $\alpha_i g(x_i^*) = 0$ for each $i = 1, 2, \dots, m$

It can be written as phase pair of conditions:

$$\begin{aligned} \alpha_i^* > 0 &\Rightarrow g_i(x^*) = 0 \\ g_i(x^*) < 0 &\Rightarrow \alpha_i^* = 0 \end{aligned}$$

(1 + 1 point)

Whenever α_i is strictly greater than zero, then this implies that the corresponding inequality constraint must hold with equality. We refer this as an active constraint. In case of SVM, active constraints are support vectors. Thus, **this applies to SVM**.

KKT Complementary requires that for any primal optimal (w^*, b^*, ζ^*) and dual optimal (α, β)

$$\alpha_i^*(1 - \zeta^* - y^{(i)}(w^{*T}x^{(i)} + b^*)) = 0$$

$$\beta^*\zeta^* = 0$$

(4 points)

For $i = 1 \dots m$, from the first condition we can see that if $\alpha_i^* > 0$ then in order for the product to be zero then $1 - \zeta^* - y^{(i)}(w^{*T}x^{(i)} + b^*) = 0$

It follows that $y^{(i)}(w^{*T}x^{(i)} + b^*) \leq 1$ Since $\zeta^* \geq 0$ by primal feasibility. Similarly if $\beta_i^* > 0$, then $\zeta^* = 0$ to ensure complementary. From the primal constraint $y^{(i)}(w^{*T}x^{(i)} + b^*) \geq 1 - \zeta_i$ it follows that $y^{(i)}(w^{*T}x^{(i)} + b^*) \geq 1$ Finally since β equivalent to $\alpha_i < C$ since $\alpha_i + \beta_i = C$ we can summarize KKT conditions. $\alpha_i^* < C \Rightarrow y^{(i)}(w^{*T}x^{(i)} + b^*) \geq 1$ $\alpha_i^* > 0 \Rightarrow y^{(i)}(w^{*T}x^{(i)} + b^*) \leq 1$

or equivalently

$$\alpha_i^* = 0 \Rightarrow y^i(W^T x^i + b^*) \geq 1$$

$$0 \leq \alpha_i^* \leq C \Rightarrow y^i(W^T x^i + b^*) = 1$$

$$\alpha_i^* = C \Rightarrow y^i(W^T x^i + b^*) \leq 1$$

Complementary slackness conditions:

$$\alpha_j^*[y_j(\bar{w}^* \cdot \bar{x}_j + b) - 1 + \zeta_j] = 0 \Rightarrow \alpha_j^* = 0 \vee y_j(\bar{w}^* \cdot \bar{x}_j + b) = 1 - \zeta_j$$

VI.10. (4 points) [END2019 Q3(c)]

Show that the constraints in a primal SVM formulation are affine, i.e., simultaneously convex and concave.

Solution: (3 points)

Primal formulation:

$$1 - \zeta^* - y^{(i)}(w^{*T}x^{(i)} + b^*) \leq 0$$

$$-\zeta \leq 0$$

$$c_1 = 1 - \zeta^* - y^{(i)}(w^{*T}x^{(i)} + b^*)$$

$$c_2 = -\zeta$$

Double Differentiate:

$$\frac{\partial c_1}{\partial w_j} = -y^i x_j^i$$

$$\frac{\partial^2 c_1}{\partial w_j^2} = 0$$

Hessian matrix would be constant positive matrix.

$$\frac{\partial^2 c_2}{\partial w_j^2} = \text{constant}$$

(1 point)

Therefore constraints are both convex and concave thus affine.

VI.11. (4 points) [END2019 Q3(d)]

How does the Lagrangian help in converting a constrained optimization problem to an unconstrained optimization problem. Starting from the original SVM formulation as a constrained optimization problem, develop the Lagrangian, and then the primal problem. Explain the equivalence of the primal problem with the original SVM formulation with constraints.

Solution: (3 points)

$$\text{minimize } \frac{1}{2} \|w\|^2$$

$$\text{Constraint : } y_i(w^T x_i + b) \geq 1$$

Primal formulation is as follows:

$$L_p(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w^T x_i + b) - 1)$$

$$\text{s.t. } \alpha_i \geq 0$$

Taking derivatives we get:

$$w = \sum_{i=1}^n \alpha_i y_i x_i \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

Applying these values in the original primal formulation we get the dual formulation.

$$\max \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

$$\text{Conditioned on } \alpha_i \geq 0 \text{ and } \sum_{i=1}^n \alpha_i y_i = 0$$

(1 point)

Equivalence from primal to dual on the dot product (because we used the lagrangian).

VI.12. (5 points) [MID2017 Q6]

(Extra credit)

- Determine the mapping $\phi(x)$ such that the kernel $k(\mathbf{x}, \mathbf{z}) = (c + \mathbf{x}^T \mathbf{z})^2 = \phi(\mathbf{x})^T \phi(\mathbf{z})$, where $\mathbf{x} = (x_1, x_2)^T$ and $\mathbf{z} = (z_1, z_2)^T$.
- Show that an XOR is not linearly separable, but that after the mapping $\phi(\mathbf{x})$ with $c = 0$, it is linearly separable.

Solution: [No solution was given in the source PDF for this question.]

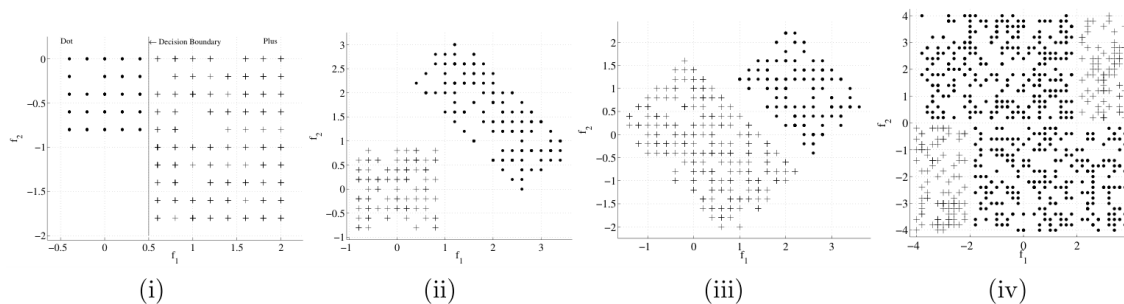
Part VII Decision Trees and Ensembles

VII.1. (5 points) [MID2017 Q1]

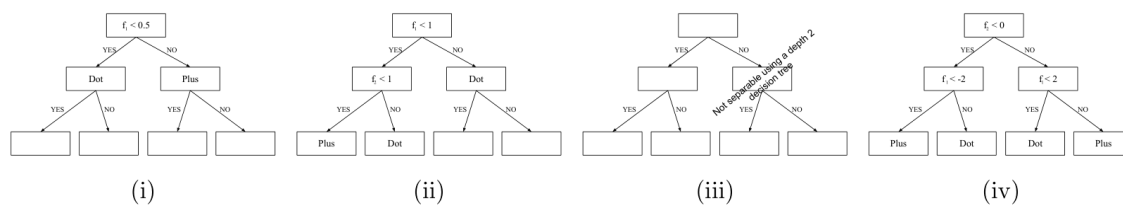
You are given points from 2 classes, shown as +’s and ·’s. For each of the following sets,

- Draw the decision tree of depth at most 2 that can separate the given data completely, where each node involves thresholding of a *single* variable.
- Draw the decision boundaries corresponding to your decision tree in your answer sheet, clearly showing the axes and slope/intercept. Write the class labels for each of the resulting regions.

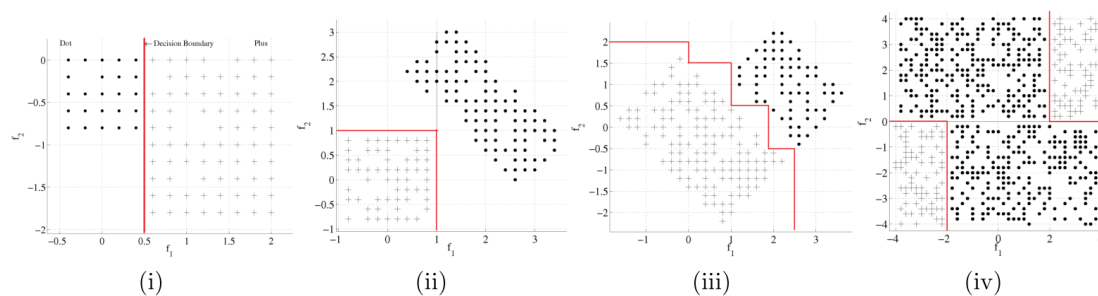
If in any of the cases, the data can not be separated completely by a depth 2 decision tree, can the data be separated using a decision tree of higher depth? If yes, can you draw one?



Solution: a)



b) In this part, sub-parts (i), (ii) and (iv) are straightforward, with the decision boundaries shown by the solid lines in the figures below. A depth two decision tree is unable to separate sub-part (iii) on the other hand and hence is not shown.



Sub-part (iii) can be determined by a higher depth tree, however, since we are only permitted to use one parameter at each node, a decision boundary is composed of horizontal and vertical boundaries partitioning the space (e.g., sub-parts (ii) and (iv)). Even though the dots and pluses are linearly separable, the linear boundary is not aligned with the horizontal or the vertical axis. Thus it will have to be approximated by a 'staircase' like boundary generated by a higher depth decision tree, which is probably too large to draw.

VII.2. (2 points) [MID2017 Q2(c)]

What effect will pruning a decision tree (to a certain depth) have on the bias and variance of your model? Mention 'increase', 'decrease' or 'no change' along with a justification supporting your choice.

Solution: Pruning decision trees also increase the bias and reduce the variance. We know that deeper decision trees tend to overfit and pruning has an effect of reducing the overfitting.

VII.3. (20 points) [END2018 Q4]

Consider the training dataset given below. In the dataset, X_1 , X_2 , and X_3 are the attributes and Y is the class variable.

Sample	X_1	X_2	X_3	Y
S1	0	0	0	+
S2	0	0	1	-
S3	0	1	0	-
S4	0	1	1	+
S5	1	0	0	-

Sample	X_1	X_2	X_3	Y
S6	1	1	0	+
S7	0	1	1	-
S8	1	1	1	-

Table 1: Left: Training set; Right: Validation set

- (5 points) Which attribute has the highest information gain? Justify your answer?
- (9 points) Draw the (full) decision tree for this dataset using the information gain criteria.
- (6 points) Can you construct a decision tree that will have 100% accuracy on the validation dataset as well as the training dataset. Answer Yes or No. If your answer is yes, then draw the decision tree that will have 100% accuracy. If your answer is no, then explain why it cannot be done. No credit will be given if the explanation is incorrect.

Solution:

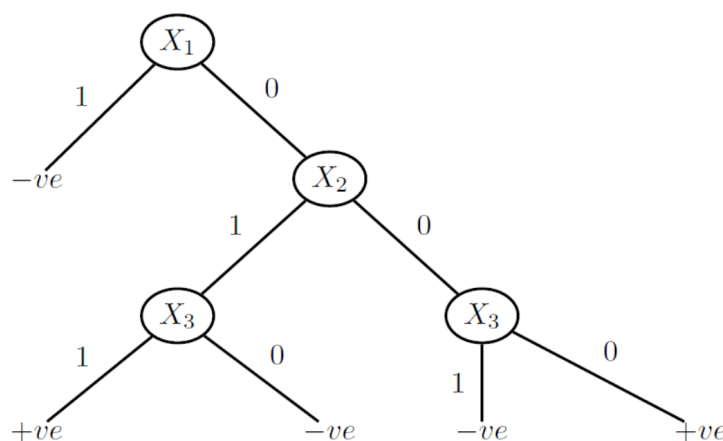


Figure: Solution for part (b). Note that X_2 and X_3 are interchangeable, as their Information Gains are the same.

(a)

$$IG(X_1) = \text{Entropy}(2^+, 3^-) - \frac{1}{5}\text{Entropy}(0^+, 1^-) - \frac{4}{5}\text{Entropy}(2^+, 2^-) = 0.1709$$

$$IG(X_2) = \text{Entropy}(2^+, 3^-) - \frac{2}{5}\text{Entropy}(1^+, 1^-) - \frac{3}{5}\text{Entropy}(1^+, 2^-) = 0.0199$$

$$IG(X_3) = \text{Entropy}(2^+, 3^-) - \frac{2}{5}\text{Entropy}(1^+, 1^-) - \frac{3}{5}\text{Entropy}(1^+, 2^-) = 0.0199$$

- (b) See the figure above.
- (c) It is not possible to achieve 100% accuracy, as samples S4 and S7 are conflicting, i.e., they have inconsistent or *noisy* labels.

VII.4. (5 points) [END2019 Q1(b)]

(1+2+2) What is entropy? Give the mathematical definition of entropy. Justify or refute the statement “A node should be a leaf node if its entropy is zero”. Can you give one (or more) scenarios where you may choose to have a leaf node that has non-zero entropy?

Solution: Entropy is defined as the expected number of bits needed to encode a randomly drawn value of a random variable X . It could also be defined as a measure of impurity, disorder or uncertainty in a set of samples. (0.5 marks + 0.5 for formulation)
Entropy $H(X)$ of a random variable X can be written as

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

Justify: The entropy at a particular node will be 0 when the dataset S at that node is perfectly classified. There would be no point in splitting it further and would hence be a leaf node.

Leaf node with non zero entropy: One case could be when the current dataset has data from multiple classes remaining, but there are no more attributes left to split the data on. Other cases could include degradation of performance due to splitting, pruning.

VII.5. (5 points) [END2019 Q1(a)]

We have seen decision trees for classification in class. How would you extend them to perform a regression task? What would be the advantage of doing so over linear regression?

Solution: We normally use entropy as a measure of impurity for classification with decision trees. For regression tasks, we can use Weighted Mean Square Error of the children nodes instead to model continuous variables.

$$MSE(t) = \frac{1}{N_t} \sum_{i \in D_t} (y^{(i)} - \hat{y}_t)^2$$

N_t is the number of samples at node t , D_t is the training subset at node t , $y(i)$ is the true target value and $\hat{y}_t$ is the predicted target value. (3 marks, 1 mark for partial correct)

Alternatively, modeling the probability of occurrence as a gaussian distribution, and calculate entropy based on this. Answers have been considered provided they are fully correct.
Advantages over linear regression: (2 marks, any one point)

- Decision trees support non linear decisions
- Decision trees, being non parametric, support a large number of functional forms
- No assumptions required about the underlying functions

VII.6. (5 points) [END2019 Q1(c)]

Give a total of five advantages and disadvantages of decision trees over classifiers like SVMs or logistic regression. (1 mark for each point)

Solution: Advantages -

- Decision trees are non parametric, and hence support a large number of functional forms
- Decision trees can be used for both continuous and categorical data
- Decision trees don't require much data processing (such as scaling, normalization, etc).
- Implicit feature selection

Disadvantages -

- Decision trees are much more prone to overfitting, prone to sampling errors
- Small changes in the data can cause a large change to the structure of the decision tree. In SVM or logistic regression classifiers, changes to the data do not affect the decision boundary as extensively
- Exponential complexity in calculation with growing datasets.
- Tree splitting is locally greedy and can lead to an incorrect feature hierarchy which may lead to incorrect classifications

VII.7. (4 points) [END2017 Q1]

Why are “weak” learners used with boosting method?

Solution: Boosting is a sequential ensemble learning method where the complexity of the overall classifier increases with each step. Since weak learners are simple learners that do not overfit, having a weak learner in each stage implies that the overall classifier is less likely to overfit.

Part VIII Neural Networks and Deep Learning

VIII.1. (6 points) [END2017 Q5(a) $\equiv$ END2019 Q4(b) $\equiv$ END2019 Q4(f), first half]

(2+2+2 points) Write the mathematical expressions of sigmoid, tan-hyperbolic, and linear rectified activation functions used in neural networks. State the pros and cons of these activation functions. Compute the derivative of each, explain why ReLU is expected to work better in deeper networks than sigmoid or tanh, and say whether a tanh activation can be used at the output layer of a binary classifier.

Solution: The **Sigmoid** activation function is defined as

$$\sigma : \mathbb{R} \rightarrow [0, 1]$$
$$z \rightarrow \frac{1}{1 + e^{-z}}$$

- Pros:** 1) It provides direct probabilistic interpretation of its output in the range of $[0,1]$ and hence, is very useful in classification and logistic regression via neural networks.
2) Its gradient can be computed very efficiently as

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

which is very helpful in neural networks for weight update via backpropagation.

Cons: Large negative inputs lead to zero output and hence, network would get stuck in the current state and no further update of weights may happen for some neurons. This leads to slow convergence and the network may not reach its optimal point.

Tan-hyperbolic activation function is defined as

$$\begin{aligned}\sigma &: \mathbb{R} \rightarrow [-1, 1] \\ z &\rightarrow \frac{e^z - e^{-z}}{e^z + e^{-z}}\end{aligned}$$

- Pros:** 1) It overcomes the problem associated with the Sigmoid activation function for negative inputs because it has an output range from $[-1,1]$ and generates an output toward -1 for large negative inputs.
2) Its gradient can be computed efficiently as

$$\sigma'(z) = (1 - \sigma^2(z))$$

Cons: Both Sigmoid and Tan-hyperbolic are not well suited for deep neural networks because these networks suffer with the problem of vanishing/exploding gradients.

Rectified linear activation function (RELU) is defined as

$$\sigma(z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0. \end{cases}$$

Pros: It addresses the problem of vanishing/exploding gradient in deep neural networks. and hence, has been shown to speed up the training process over tanh and sigmoid by approx. six times.

Cons: A major disadvantage of RELU is the zero gradient for negative inputs that de-activates a neuron for negative input. This can lead to many de-activated neurons in the network that shows its impact adversely on convergence of the network to optimal solution.

Alternative solution (END2019 Q4(b) – tanh in more detail, and tanh at the output layer):

$$\text{Derivative of tanh:} \quad \tanh' = 1 - \tanh(x)^2 \quad (1)$$

- Like sigmoid, tanh is also a sigmoidal -(S-shaped) function.
- The derivative of tanh includes the tanh function itself, the caching trick (same as sigmoid) can be used for layers that implement tanh activation functions.
- The derivatives are steeper for tanh than sigmoid.
- Sigmoid maps strong-negative inputs to values close to zero, because of this neural network get stuck during training. While, tanh maps strongly negative inputs to the negative outputs and only zero-valued inputs are mapped to near-zero outputs. These properties make the network less likely to get stuck during training.

Yes, we can use tanh activation at the output layer of binary classification.

Alternative solution (END2019 Q4(f), first half – ReLU in deep networks): The Rectified Linear Unit is the most commonly used activation function. The function returns 0 if it receives any negative input, for any positive value x it returns that value back. (1 marks)

The main problem with Sigmoid and tanh is that they saturate. During backpropagation, they may produce a gradient of zero for large inputs, which can slow or even halt the learning process. The ReLU function $f(x) = \max(0, x)$ doesn't saturate for large positive inputs.

The ReLU function has a derivative of 0 over half it's range (the negative inputs). For positive inputs, the derivative is 1. When training on a reasonable sized batch, there will usually be some data points giving positive values to any given node. So the average derivative is rarely close to 0, which allows gradient descent to keep progressing. (2 marks)

VIII.2. (5 points) [END2019 Q4(a)]

Give the expression for binary cross-entropy (BCE) loss. Usually, it is used with a sigmoid output activation. Argue why is the BCE loss more effective than the mean-squared error (MSE) loss. Make use of corresponding gradients to support your arguments.

Solution: Expression for BCE

$$BCE = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i)) \quad (1)$$

(1 marks)

If we use BCE instead of MSE loss, learning can occur much faster during the beginning phases of training.

Proof: Consider we have a binary classification setup, where input feature is x_i , ground truth label is y_i and $\hat{y}_i = h_{\theta}(x_i) = \sigma(Wx_i + b)$ is the corresponding prediction.

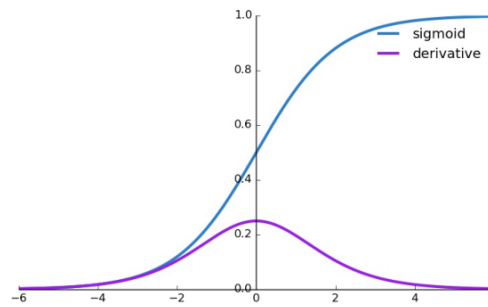
$$\text{MSE LOSS : } J = \frac{1}{2}(y_i - \hat{y}_i)^2 \quad (2)$$

$$\frac{\partial J}{\partial W} = (y_i - \hat{y}_i)\sigma(Wx_i + b)(1 - \sigma(Wx_i + b)) \quad (3)$$

(1 marks)

Since we initialized our weights randomly with values close to 0, this expression in equation (3) will be very close to 0, which will make the partial derivative nearly vanish during the early stages of training. As you can see in the figure below, gradients are small whenever outputs are close to 0 or 1. This can lead to slower learning at the beginning stages of gradient descent, since the smaller derivatives change each weight by only a small amount, and gradient descent takes a while to get out of this loop and make larger updates towards a minima. (1 marks)

On the other hand, if we use BCE loss:



$$\text{BCE LOSS} : -\frac{1}{N} \sum_{i=1}^N y_i \log(\sigma(Wx_i + b)) + (1 - y_i) \log(1 - \sigma(Wx_i + b)) \quad (4)$$

$$\frac{\partial J}{\partial W} = x_i(\sigma(Wx_i + b) - y_i) \quad (5)$$

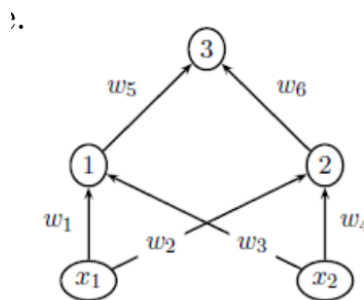
(1 marks)

We can see from equation (5) the magnitude of gradient completely depends on the magnitude of the error $(\sigma(Wx_i + b) - y_i)$ —i.e how far our prediction is from the ground truth. It means that during early phase of learning, the error magnitude would be larger, hence fast learning, while later on magnitude would be smaller which leads of smaller derivatives, and hence corresponding to smaller adjustments to the weight variables.

(1 marks)

VIII.3. (10 points) [END2018 Q3(b)]

Consider the neural network shown in the figure below. Assume that all internal nodes and output nodes compute the $\tanh()$ activation function. Using the squared error loss, derive the weight update equations using backpropagation for $(w_1, \dots, w_6)$. {Hint: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$; Try simplifying the gradient of $\tanh()$ as you would do for sigmoid.}



Solution: We will use the identity, $\frac{\partial}{\partial z}(\tanh(z)) = 1 - \tanh^2(z)$. Let the input *logit* to each node be denoted by z_i , and the output of the activation be denoted by $\hat{y}_i = \tanh(z_i)$, where the index $i = 1, 2, 3$ identifies the neuron (number written in the circle in the figure).

The loss is given as $L = (\hat{y}_3 - y)^2$, where y is the ground truth value of the prediction. Now, we have the following relations

$$\frac{\partial \hat{y}_i}{\partial z_i} = 1 - \tanh^2(z_i), \quad i = 1, 2, 3$$

$$z_1 = w_1x_1 + w_3x_2$$

$$z_2 = w_2x_1 + w_4x_2$$

$$z_3 = w_5\hat{y}_1 + w_6\hat{y}_2$$

The corresponding partials are given by

$$\frac{\partial L}{\partial \hat{y}_3} = 2(\hat{y}_3 - y)$$

$$\begin{aligned} \frac{\partial z_3}{\partial \hat{y}_1} &= w_5 & \frac{\partial z_3}{\partial \hat{y}_2} &= w_6 & \frac{\partial z_2}{\partial w_2} &= x_1 & \frac{\partial z_2}{\partial w_4} &= x_2 \\ \frac{\partial z_1}{\partial w_1} &= x_1 & \frac{\partial z_1}{\partial w_3} &= x_2 & \frac{\partial z_3}{\partial w_5} &= \hat{y}_1 & \frac{\partial z_3}{\partial w_6} &= \hat{y}_2 \end{aligned}$$

Now, by applying the chain rule, we can obtain the update equations for each of the parameters. The elements of the gradient vector would be $\frac{\partial L}{\partial w_i}, i = 1, \dots, 6$.

$$\begin{aligned} \frac{\partial L}{\partial w_6} &= \frac{\partial L}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial z_3}{\partial w_6} \\ \frac{\partial L}{\partial w_5} &= \frac{\partial L}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial z_3}{\partial w_5} \\ \frac{\partial L}{\partial w_4} &= \frac{\partial L}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial \hat{y}_2}{\partial z_2} \frac{\partial z_2}{\partial w_4} \\ \frac{\partial L}{\partial w_3} &= \frac{\partial L}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial w_3} \\ \frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial \hat{y}_2}{\partial z_2} \frac{\partial z_2}{\partial w_2} \\ \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial w_1} \end{aligned}$$

VIII.4. (6 points) [END2017 Q5(b)]

(2+2+2 points) Given a neural network in Figure-2 below:

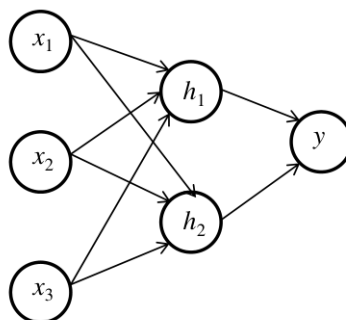


Figure 2: Neural Network

We use linear rectified activation function σ at the hidden layer and output layer neurons. The loss function is given by $L = \frac{1}{2}(y - t)^2$, where t denotes the target value. Let us denote the hidden layer and output layer weights by matrices $\mathbf{W}$ and $\mathbf{V}$, respectively, that are initialized as below:

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{V} = \begin{bmatrix} 0 & 1 \end{bmatrix}. \quad (1)$$

- (i) Write the expression of mapping x to y in terms of σ , $\mathbf{W}$, and $\mathbf{V}$.
- (ii) Assume that the current input is $\mathbf{x} = [1 \ 2 \ 1]$. The target value is $t = 1$. Compute the numerical output value y . Show all intermediate steps.
- (iii) Compute the gradient of the loss function with respect to the weights $\mathbf{V}$ and $\mathbf{W}$. Also, compute the values numerically for the choices of $\mathbf{W}$, $\mathbf{V}$, $\mathbf{x}$, and t given above.

Solution:

[Handwritten solution, reproduced from the scanned source pages.]

Solution (i) Let $\underline{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{3 \times 1} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \leftarrow \text{Given}$

$\underline{h} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}_{2 \times 1}$ $\mathbf{V}_{1 \times 2} = \begin{bmatrix} 0 & 1 \end{bmatrix} \uparrow \text{given}$

Given $\sigma(z) = \max(0, z) \neq \sigma_1(z)$

$\underline{h}_{2 \times 1} = \sigma_1 \left(\mathbf{W}_{2 \times 3} \underline{x}_{3 \times 1} \right)$ $\uparrow$ hidden layer

$\sigma_2(z)$ $\uparrow$ o/p layer

$y = \sigma_2 \left(\mathbf{V} \underline{h} \right) = \sigma_2 \left(\mathbf{V} \sigma_1 \left(\mathbf{W} \underline{x} \right) \right)$

where $\sigma_1 = \sigma_2 = \sigma$

$y = \sigma \left(\mathbf{V} \sigma \left(\mathbf{W} \underline{x} \right) \right)$

Part (i).

$$(ii) \quad \underline{w}x = \begin{pmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix} \quad \underline{3}$$

$$\underline{h} = \sigma_1 \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} \max(2, 0) \\ \max(-1, 0) \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

$$V_h = (0, 1) \begin{pmatrix} 2 \\ 0 \end{pmatrix} = 0$$

$$y = \sigma(0) = 0$$

$$(iii) \quad \frac{\partial L}{\partial v} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial v}$$

$$L = \frac{1}{2} (y - t)^2$$

$$\frac{\partial L}{\partial y} = (y - t) \quad 1 \times 1$$

$$\frac{\partial y}{\partial v} = \sigma_2' (V_h) \frac{\partial V_h}{\partial v} \quad 1 \times 1$$

$$\therefore \frac{\partial L}{\partial v} = \left(\frac{\partial L}{\partial y} \circ \sigma_2' (V_h) \right) \frac{\partial V_h}{\partial v}$$

↑
hadamard product

$$\sigma_2' (V_h) = \frac{\partial \sigma_2 (V_h)}{\partial V_h} = 0 \text{ or } 1$$

↑
Subgradient method

Compute gradient for $z \leq 0$ & $z > 0$ separately

$$z \leq 0$$

$$\text{gradient} = 0$$

$$z > 0$$

$$\text{gradient} = \frac{\partial z}{\partial v} = 1$$

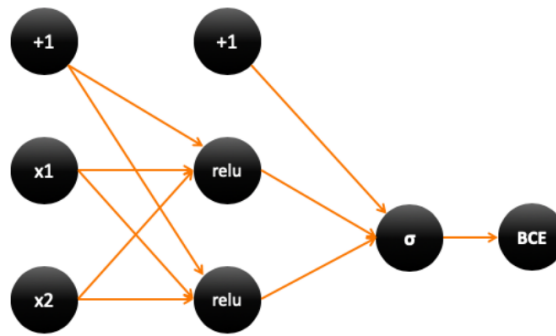
$$\begin{aligned}
 \therefore \frac{\partial L}{\partial V} &= \underset{1 \times 1}{(y-t)} \underset{1 \times 1}{g_2} \frac{\partial V h}{\partial V} \\
 &= (y-t) g_2 h^T \\
 \begin{bmatrix} \frac{\partial L}{\partial v_1} & \frac{\partial L}{\partial v_2} \end{bmatrix}_{1 \times 2} &= \underset{1 \times 2}{(y-t) g_2 h^T} = -1 g_2 \begin{bmatrix} 2 & 0 \end{bmatrix} \\
 &\quad \text{but } g_2 = 0 \text{ for given values} \therefore \frac{\partial L}{\partial V} = \begin{bmatrix} 0 & 0 \end{bmatrix} \\
 \hline
 \frac{\partial L}{\partial W} &= \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial W} \\
 &= \left(\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial V h} \right) \frac{\partial V h}{\partial W} \\
 &= \left(\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial V h} \right) \frac{\partial V h}{\partial h} \cdot \frac{\partial h}{\partial W} \\
 &= \left(\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial V h} \right) \left(\frac{\partial V h}{\partial h} \cdot \frac{\partial h}{\partial W x} \right) \frac{\partial W x}{\partial W} \\
 &\quad \begin{matrix} \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 \times 1 & 1 \times 1 & 2 \times 1 & 2 \times 1 & 1 \times 3 \\ (y-t) & g_2' & v^T & g_1 & x^T \end{matrix} \\
 &= (y-t) g_2 (v^T g_1) x^T \\
 &= (-1) \times 0 \times \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} g_{11} \\ g_{12} \end{bmatrix} \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \\
 &\quad \begin{matrix} \text{--- } 2 \times 1 & 2 \times 1 & 1 \times 3 \end{matrix} \\
 &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}
 \end{aligned}$$

Part (iii), page 2.

VIII.5. [QUIZ3 Q4]

Consider a neural network with only one hidden layer comprising of two neurons, each having *relu* activations ($\text{relu}(z) = \max(0, z)$). Assume that the neural network is performing a binary classification, implying the output layer has a single neuron with a sigmoid activation. Draw this neural network for two dimensional input data. Derive the gradient update rule for each of the weights using backpropagation.

Solution: Given, two features in input layer (x_1, x_2), one hidden layer with two neurons (h_1, h_2) using *relu* as activation. Output layer with one neuron (o) using sigmoid activation. After adding the bias terms (x_0, h_0) the architecture looks like the following.



$$\text{relu}(x) = \{x \text{ for } x > 0; \text{ otherwise } 0\}$$

$$\text{relu}'(x) = \{1 \text{ for } x > 0; \text{ otherwise } 0\}$$

$$\sigma(x) = \frac{1}{1 + e^x}$$

$$\sigma'(x) = \sigma(x) * (1 - \sigma(x))$$

Forward Propagation

$$h_1 = \text{relu}(x_0 * w_{01}^1 + x_1 * w_{11}^1 + x_2 * w_{21}^1)$$

$$h_2 = \text{relu}(x_0 * w_{02}^1 + x_1 * w_{12}^1 + x_2 * w_{22}^1)$$

$$O = \sigma(h_0 * w_{01}^2 + h_1 * w_{11}^2 + h_2 * w_{21}^2)$$

Backpropagation

For updating the weights we need not compute the loss as we will use only the derivatives of the loss.

$$E = -\frac{1}{m} \sum_1^m (y_i * \log(O_i) + (1 - y_i) * \log(1 - O_i))$$

Using chain rule we can calculate the derivatives for backpropagation.

Second layer weights

$$\frac{\partial E}{\partial w^2} = \frac{\partial E}{\partial O_{out}} * \frac{\partial O_{out}}{\partial O_{in}} * \frac{\partial O_{in}}{\partial w^2}$$

$$\frac{\partial E}{\partial O_{out}} = -1(y_i * \frac{1}{O_{out}} + (1 - y_i) * \frac{1}{1 - O_{out}})$$

$$\frac{\partial O_{out}}{\partial O_{in}} = \sigma'(O_{in})$$

$$\frac{\partial O_{in}}{\partial w^2} = h_i$$

Let η be the learning rate.

$$w^2 = w^2 - \eta * \frac{\partial E}{\partial w^2}$$

First layer weights

$$\frac{\partial E}{\partial w^1} = \frac{\partial E}{\partial h_{out}} * \frac{\partial h_{out}}{\partial h_{in}} * \frac{\partial h_{in}}{\partial w^1}$$

$$\frac{\partial E}{\partial h_{out}} = \frac{\partial E}{\partial w^2}$$

$$\frac{\partial h_{out}}{\partial h_{in}} = \text{relu}'(h_{in})$$

$$\frac{\partial h_{in}}{\partial w^1} = x_i$$

$$w^1 = w^1 - \eta * \frac{\partial E}{\partial w^1}$$

VIII.6. (10 points) [END2018 Q3(a) $\equiv$ QUIZ4 Q3]

What are the different components of a *deep* Convolutional Neural Network applied to classification? Describe each component based on its need, functionality and advantages over traditional neural network architectures (multi-layer perceptrons).

Solution:

- (i) *Convolutional Layer*: Need to have multiple representations, but only based on local patterns to maintain a small number of network parameters.
- (ii) *Rectified Linear Unit (ReLU)*: Need to propagate gradients without reducing their magnitude (unlike sigmoid/tanh or other saturating activations), and is required for deep networks with many layers.
- (iii) *Pooling Layer*: Pooling layer effectively subsamples the feature maps from the previous layer, in turn increasing the spatial footprint (receptive field) for the same convolutional kernel size. It helps in reducing the number of parameters and allows the network capture more complex pattern spread across a larger spatial extent.
- (iv) *Fully connected*: Typically, these show up in the last few layers of a CNN, where the high-level features are collected to make the final classification decision. Usually, they feed into the softmax layer.

Alternative solution (QUIZ4 Q3 – stated as drawbacks of MLPs and advantages of CNNs): Drawbacks of MLP -

- Little or no invariance to shifting, scaling, and other forms of distortion
- The topology of the input data is completely ignored
- Fully connected hidden layer would have an unmanageable number of parameters
- Computing the linear activations of the hidden units would be very expensive.

Advantages of CNN

- Local Connectivity - Each hidden unit is connected only to a subregion (patch) of the input image, therefore it takes local structure/connectivity into account.

- **Parameter Sharing** - Units organized into the same “feature map/template” share parameters. Reduces total number of parameters. This also ensures that all neurons detect the same feature at different positions in the input image, thus ensuring Feature Equivariance.
- **Stride** (typically denoted by s in CNNs) decides the spacing between overlapping convolutions - Helps in reducing parameters further.
- **Sub-Sampling/Pooling layers** - The subsampling layers reduce the spatial resolution of each feature map. By reducing the spatial resolution of the feature map, a certain degree of shift and distortion invariance is achieved.

VIII.7. [QUIZ4 Q4]

Why are the convolution and pooling layers alternately placed in a typical CNN architecture? Could you achieve a similar effect as pooling by changing a hyperparameter like stride?

Solution: Convolutional layers detect patterns that are locally present in a region. By alternating with pooling layers, we effectively *increase the receptive field of the neurons*, thus allowing the neurons to detect bigger, more complex patterns. By subsampling, some adverse effects of noise and distortion in input image and patterns could be reduced.

The same effect of a larger receptive field could potentially be achieved by stacking together more convolutional layers, however, the number of parameters would end up being much more. Another alternative to pooling is to increase the stride hyperparameter of the convolution layer.

VIII.8. (5 points) [END2019 Q4(d)]

The first layer of AlexNet has 96 filters of size 11×11 , that process an input image of $224 \times 224 \times 3$ to generate feature maps of size 55×55 . What is the stride used in the first layer? The last max pooling layer of size $(13 \times 13 \times 256)$ is connected to a two layered fully-connected layer of size 4096, followed by a 1000-node softmax based output layer. What is the total number of parameters for the last four layers (max pooling, linear, linear and softmax)?

Solution: Stride : 3.94 Assuming $224 \times 224 \times 3 / 4$ assuming $227 \times 227 \times 3$

(1 marks) No. of parameters (max pool) : 0

(1 marks) No. of parameters (FC1) : 37,752,832

(1 marks) No. of parameters (FC2) : 16,781,312

(1 marks) No. of parameters (softmax) : 4,097,000

VIII.9. (4 points) [END2019 Q4(f), second half $\equiv$ END2019 Q5(b)]

What is dropout, and why is it effective in reducing overfitting? What other strategies would you use to avoid overfitting in deep neural networks?

Solution: Dropout is a technique for addressing the overfitting problem. The key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much. During training, dropout samples from an exponential number of different “thinned” networks. At test time, it is easy to approximate the effect of averaging the predictions of all these thinned networks by simply using a single unthinned network that has smaller weights. This significantly reduces overfitting and gives

major improvements over other regularization methods. (2 marks)
For detail please refer [this](#) document.

Alternative solution (END2019 Q5(b) – the wider list): Write any two points: (1 mark for each point)

- Adding dropout layers.
- Feature selection
- Residual connections
- Data augmentation
- Regularization / Early stopping

Part IX Unsupervised Learning and Dimensionality Reduction

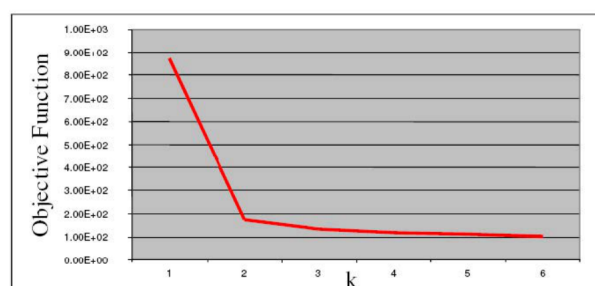
IX.1. (2 points) [MID2016 Q1]

What is the objective function of k -means clustering? How would you choose an appropriate value of k ?

Solution: The k -means objective function is:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|_2^2 \quad (1)$$

An appropriate value of k can be chosen by plotting J in (1) by varying k and identifying a value of k which leads to a large drop in J followed by a ‘flatish’ region. An example is shown below:



Selection of k in k -means. The cusp, i.e., the steep drop (at $k = 2$ in this image), followed by a flat region indicates a reasonable choice.

IX.2. (5 points) [MID2016 Q3]

Write the expression for the pdf arising from a K mixture of gaussians. Given an i.i.d. sample of N data points, what will be the log likelihood function that you would maximize

to estimate the parameters? Derive the expression for the mixing coefficients π_k . If the k^{th} Gaussian tends to collapse to a single point ($\pi_k \rightarrow 1/N$, $\boldsymbol{\mu}_k \rightarrow \mathbf{x}_i$, $\boldsymbol{\Sigma}_k \rightarrow \mathbf{0I}$), will the likelihood function value tend to increase? Explain why or why not. Given your goal is to model the data using a mixture of Gaussians, comment on what needs to be done, if anything at all.

Solution: The pdf arising from mixture of K gaussians can be written as:

$$p(\mathbf{x}_i | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Using above pdf and for a data set containing N i.i.d. data points, the likelihood function can be written as

$$p(\mathbf{X} | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \prod_{i=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Taking logarithm both sides,

$$\ln(p(\mathbf{X} | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})) = \sum_{i=1}^N \ln \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

To maximize the log-likelihood with respect to the mixing coefficients, we must take into account the constraint, $\sum_{k=1}^K \pi_k = 1$. This can be achieved by using a lagrange multiplier and maximizing the following quantity

$$\begin{aligned} \mathcal{L}(\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) &= \ln(p(\mathbf{X} | \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \\ &= \sum_{i=1}^N \ln \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \end{aligned}$$

Differentiating above equation with respect to π_k

$$0 = \sum_{i=1}^N \frac{\mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} + \lambda$$

Recall that,

$$\begin{aligned} \gamma_{nk} &= \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \\ \Rightarrow 0 &= \sum_{i=1}^N \frac{\gamma_{nk}}{\pi_k} + \lambda \end{aligned}$$

Multiplying both sides by π_k and then summing up over all values of k results in $\lambda = -N$.

$$\begin{aligned} \Rightarrow \sum_{i=1}^N \frac{\gamma_{nk}}{\pi_k} &= N \\ \Rightarrow \pi_k &= \sum_{i=1}^N \frac{\gamma_{nk}}{N} = \frac{N_k}{N} \end{aligned}$$

If the k^{th} gaussian tends to collapse to a single point then the log-likelihood would increase because then the single gaussian component can be written as

$$\mathcal{N}(\mathbf{x}_i|\mathbf{x}_i, \sigma_k^2 I) = \frac{1}{(2\pi)^{\frac{1}{2}} \sigma_k}$$

As $\sigma_k \rightarrow 0$, the above term goes to infinity and hence the log-likelihood. Likelihood function gets larger as we add more components (and hence parameters) to the model.

To model the data using a gaussian mixture model, the singularities needs to be avoided otherwise the log-likelihood function will seem to be larger but in reality, the data is fitted only by few gaussians. Perturbing the mean and covariances as the Gaussian collapses is one strategy. Running EM algorithm multiple times is another one.

IX.3. [QUIZ4 Q2]

Principal Component Analysis (PCA) is powerful tool for dimensionality reduction. It finds a linear mapping $x_{low} = Wx_{high}$ such that the inverse mapping $\hat{x}_{high} = W'x_{low}$ has a small ℓ_2 error with respect to the original x_{high} . The lower-dimensional representation is often used for ML tasks like classification, etc. Can you design the simplest neural network that is capable of implementing PCA exactly? Give the architecture (number of layers, activation, etc.) and the loss function that you would use.

Solution: We want to learn an encoder function that has a transformation, W that maps $x_{high} \in \mathbb{R}^D$ to $x_{low} \in \mathbb{R}^d$, and the decoder to learn an inverse mapping that minimizes the ℓ_2 error. The encoder should be a neural network that applies a linear transformation W that has dimensionality $d \times D$, and the decode would apply a linear transformation $W' \in \mathbb{R}^{D \times d}$. Therefore, I will have a single layer auto-encoder with linear activations. The number of input neurons will be equal to the dimensionality of x_{high} and the number of neurons in the hidden (or latent) layer would be that of x_{low} . The loss function used would be the following

$$L(W, W') = \frac{1}{N} \sum_{i=1}^N \|x_{high}^{(i)} - W' (W x_{high}^{(i)})\|_2^2 \quad (1)$$

Part X Model Evaluation, Bias–Variance and Cross-Validation

X.1. (4 points) [MID2018 Q3(a)]

Write mathematical formulation of one performance evaluation metric for each of the following tasks.

(i) Classification, (ii) Regression

Solution: For classification (2 marks)

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

TP = Number of True Positives

FP = Number of False Positives

TN = Number of True Negatives

FN = Number of False Negatives For regression (2 marks)

Root Mean Square Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}$$

y_i denotes the true score for the i-th data point, and $\hat{y}_i$ denotes the predicted value.

X.2. (4 points) [MID2018 Q3(b)]

What are the drawbacks of holdout set? Also, suggest an alternate method.

Solution: The disadvantage of using holdout set is that the evaluation can have a high variance as it may depend heavily on which data points end up in the training set and which end up in the test set. Thus, the evaluation may be significantly different depending on train-test split. (2 marks)

K-fold cross validation is one way to improve over the holdout method. (1 mark)

The advantage of this method is that it matters less how the data gets divided. Every data point gets to be in a test set exactly once, and gets to be in a training set k-1 times. Since the validation error is averaged over N/k different validation samples, the variance in the error estimates would be reduced. (1 mark)

X.3. (4 points) [MID2018 Q3(c)]

How is K-fold cross validation different from Leave-one-out Cross validation? (write any two differences). If the data is sparse, which cross-validation method would be beneficial?.

Solution: In leave-one-out cross validation, $K = \text{Number of datapoints (N)}$ ie. training is done all the data except for one point. It requires more computation time than K-fold cross validation since N training cycles are required. (1 mark each for the 2 differences)

Leave-one-out cross validation is better for sparse data (1 mark)

Since the data is very sparse, leave-one-out CV will allow us to train on as many samples as possible. (1 mark)

X.4. (4 points) [MID2018 Q3(d)]

Write two pros and cons of choosing (i) Large number of folds, (ii) Small number of folds in a cross-validation task:

Solution: Larger number of folds

Pros:

(a) The bias of the true error rate estimator will be small.

Cons:

(a) The variance of the true error rate estimator will be large.

(b) The computational time will be very large.

Smaller number of folds

Pros:

(a) The computation time are reduced as there will be lesser number of experiments.

(b) The variance of the estimator will be small as each experiment has a larger number of validation samples.

Cons:

(a) The bias of the estimator will be larger.

X.5. (4 points) *[MID2018 Q3(e) $\equiv$ END2019 Q5(c)]*

Explain how would you detect (i) High variance, (ii) High bias, in your learning model. Write one solution each for fixing them.

Solution: High Variance

Detection (1 mark)

The errors over the training set and validation sets would be very different in case of high variance. This may indicate overfitting of your model on your training set.

Solution (1 mark)

More training examples, smaller set of features. (any one)

High Bias

Detection (1 mark)

If the cross-validation and training errors are similar for a range of training set sizes, and yet they are significantly larger than what is expected, then the model has high bias. It indicates that the model is underfitting.

Solution (1 mark)

Larger set of features, or a higher capacity model.

Alternative solution (END2019 Q5(c) – a checklist for underfitting):

- Check if the training error is quite high.
- Check if the difference between the training and test errors is low. (2 marks)
- Does the performance remain the same upon increasing the data size?
- Does changing / increasing the feature set help in refining the performance?

X.6. (1 point) *[MID2021 Q2(a) $\equiv$ END2018 Q2(b)]*

You are given a data set for detecting a rare type of cancer. You've build a classification model and achieved a validation accuracy of 96%. Why shouldn't you be happy with your model performance? Is accuracy the right metric for this problem? Justify your argument {*Hint: What is the fraction of people who have this type of cancer as opposed to people who do not?*}

Solution: Rare cancer detection lies in the category of imbalanced class problems, where the class distributions are highly skewed. The fraction of people who have this rare type of cancer (Class A) is very low in comparison to people who don't (Class B). Suppose we have 4% instances in class A, and the rest 96% in class B. Accuracy is not the best metric to use in such cases, because a dumb classifier, which classifies all the samples to Class B, would end up getting a really high accuracy of 96%, even when it does not make correct predictions on the minority class.

Alternative solution (END2018 Q2(b), where the same situation is posed for a soft-margin SVM trained on data that is 95% class 1 and that reaches 95% test accuracy): Simply looking at the accuracy, it is not clear if the SVM has learned a meaningful boundary. It might be classifying all the points as class 1. We need to analyze the misclassification errors.

X.7. (1 point) *[MID2021 Q2(b)]*

You are working on a classification problem. For validation purposes, you've randomly partitioned the training data into a train subset and a validation subset. You are confident

that your model will work incredibly well on unseen data since your validation subset accuracy is high. However, you get shocked after getting poor test accuracy. What could have gone wrong? How would you resolve it?

Solution: The most likely explanation for degradation on test set performance is distribution shift i.e. the test and the train data come from distributions that differ significantly. One can resolve this by obtaining training data that more closely resembles the test data i.e comes from a distribution very similar to that of the test data. Another possible but less likely explanation for such degradation in performance is overfitting to the dev set. In this case we can resolve this by discarding the old dev set and creating a fresh one.

X.8. (2 points) [MID2021 Q2(c)]

Assume that you have a large dev set on which your classifier achieves a 20% error. Describe using examples, how would you use *error analysis* to improve your classifier's performance. *Hint:* Is there a way to discover potential sources of error?

Solution:

- Error analysis refers to the process of examining dev set examples that your model misclassified, so that you can understand the underlying causes of the errors.
- Since, the dev set is large we should randomly split it into two parts, a small eyeball dev set and a larger blackbox dev set. We would expect the model to have similar rates on both subsets of the original dev set since these were randomly created.
- Since, we do not manually look at the blackbox dev set we can use it to tune our hyper parameters or choose between different models.
- Inspecting misclassified examples from the eyeball dev set will give us insight into which class/type of images our model is struggling to classify properly. If the model is found to be struggling in classifying some subset of the classes (For example, the model has very low accuracy on the class dog) or in particular types of examples (co-occurrence of objects from 2 or more classes) then we can decide to focus our efforts towards improving performance on those classes/ types of examples.
- Additionally, we might discover some mislabelled examples in the dev set along the way and can choose to fix them as well.

Part XI Sequential Decision Making

XI.1. (6 points) [END2017 Q7]

(6=1+1+1+1+2 points) Long after winter has come and gone, the world of Game of Thrones has transitioned from barbaric to capitalistic. Gilly's son, Sam, is growing up north of the wall and has left home with a 100 pounds, seeking his own adventures. The whitewalkers (WW) led by the night king lost the war and are managing survival by stealing money from young kids. Sam has a 20% chance of running into WW. If the WW find Sam, he loses all his money. Let's assume that Sam's reward function is $R(x) = \log(1 + x)$. If WW find him, his utility is $R(0) = \log(1 + 0) = 0$, and if they don't, then it is $R(100) = \log(1 + 100)$.

- a) If you have no control over Sam's actions, what is the expected reward for Sam?
- b) LittleFinger offers theft insurance to Sam: If Sam pays 30 pounds, then LittleFinger reimburses the entire 70 pounds in case of theft (note that Sam has paid 30 pounds as premium already, so only has a balance of 70 pounds). What is the expected reward for Sam if he takes this insurance? If the goal is to maximize the reward, should Sam take the insurance?
- c) In the previous scenario, from LittleFinger's point of view, does it make sense to sell this insurance? What is the expected profit that LittleFinger would make?
- d) If Sam were to take actions through a deterministic policy, the value function for the policy would be

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') (R(s, \pi(s), s') + \gamma V^\pi(s'))$$

Define stochastic policy and write the expression for value function under such a policy.

- e) Since Sam is new to RL, affirm or refute his argument of "If the only difference between two MDPs is the discount factor γ , then they must have the same policy" with a justification or counter-example.

Note: some numbers that you may need $\log(101) \simeq 4.6151$, $\log(71) \simeq 4.2627$.

Solution:

- a) Sam's expected reward is given as: $\mathbb{E}[R] = 0.8 \times \log(101) + 0.2 \times \log(1) = 0.8 \times \log(101) + 0.2 \times 0 \simeq 3.6921$
- b) When taking insurance, Sam's expected reward equals $\mathbb{E}[R_I] 0.8 \times \log(1 + 70) + 0.2 \times \log(1 + 70) = \log(71) = 4.2627$. Yes, Sam should take the insurance because his expected reward is higher.
- c) The expected profit for LittleFinger equals $0.8 \times 30 + 0.2 \times (-40) = 16$ pounds.
- d) Define the stochastic policy as $\pi(s, a) = P(a|s)$ is the probability of taking action a when in state s . Then, the equivalent value function is written as

$$V^\pi(s) = \sum_a \pi(s, a) \sum_{s'} T(s, a, s') (R(s, a, s') + \gamma V^\pi(s'))$$

- e) A counterexample suffices to show that Sam's argument is incorrect. Consider an MDP with two sink states. Transitioning into sink state A gives a reward of 1, transitioning into sink state B gives a reward of 10. All other transitions have zero rewards. Let A be one step North from the start state. Let B be two steps South from the start state. Assume actions always succeed. Then if the discount factor $\gamma < 0.1$ the optimal policy takes the agent one step North from the start state into A, if the discount factor $\gamma > 0.1$ the optimal policy takes the agent two steps South from the start state into B.

XII.1. (20 points) [MID2018 Q1]

Answer the following True/False questions. An answer is true if it is *always* true. You have to justify your answer to get full credit.

- (a) (2 points) True or False ?.
Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$ be the two arbitrary matrices.
(i) $\text{rank}(AB) \geq \min(\text{rank}(A), \text{rank}(B))$. (ii) if $m = n$, $\text{rank}(A) = \text{rank}(A^{-1})$.
- (b) (2 points) As number of samples increases, the MAP and MLE estimate become the same.
- (c) (2 points) Naive Bayes can only classify linearly separable data.
- (d) (2 points) Naive Bayes can handle only discrete valued variables.
- (e) (2 points) In logistic regression, there exists a closed form solution for the parameters that maximizes the conditional log likelihood.
- (f) (2 points) Given a linearly separable data, the margin of the decision boundary produced by SVM will always be greater than or equal to the margin of the decision boundary produced by any other hyperplane that perfectly classifies that data.
- (g) (2 points) If my training error is too high, I should collect more training data to resolve this problem.
- (h) (2 points) If my model is overfitting, I should collect more data to fix it.
- (i) (2 points) If my features/observations/input variables are all independent, it would not matter whether I use a Bayes Classifier or a Naive Bayes classifier.
- (j) (2 points) Normalizing data is for kids. Real experts work with raw data!

Solution:

- (a) I. False; Since the column vectors of AB are all linear combinations of column vectors of A , the rank of AB is upper bounded by $\text{rank}(A)$. Similarly, the row vectors of AB are linear combinations of row vectors of B , the rank of AB is also upper bounded by $\text{rank}(B)$. Therefore, we have $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$
II. True; If the inverse exists, then the matrix A is nonsingular and therefore $\text{rank}(A) = \text{rank}(A^{-1}) = m = n$
- (b) True. In MAP, we are also considering the prior term in contrast to MLE. Consider the log-posterior expression, the log-likelihood is a summation over all n data points, while the log-prior term is only defined on the parameters. As $n \rightarrow \infty$, the log-prior term will be insignificant compared to the likelihood term.
- (c) False; In general, Naive Bayes classifier is not linear, as the decision boundary depends on underlying probability distribution. However, there may be distributions that result in linear boundaries, e.g., exponential family distributions.
- (d) False; Again, as Naive Bayes classifiers rely on underlying probability distributions, they can handle both Discrete and Continuous valued variables.
- (e) False; Because of the logit transformation, there is no closed-form solution for maximizing the log-likelihood for logistic regression. Usually iterative optimization techniques like gradient descent are used for fitting the model.
- (f) True; Since the SVM objective maximizes the minimum distance of the points from the decision boundary, the margin width will be greater than equal to any other linear classifier (hyperplane) for the given training dataset.

- (g) False; If the training error is already too high, increasing the training data will likely increase the error. High training error may imply that the learning algorithm was not run long enough, or that the model does not have enough capacity, e.g., if you are trying to fit a line in data that lies along a higher order polynomial .
- (h) True; Adding more diverse training data should help. Other strategies like regularization that may help in mitigating overfitting.
- (i) True; the Naive assumption in Naive Bayes classifier is the independence of variables. If the variables are actually independent, the Bayesian classifier works the same as naive bayes.
- (j) False; Normalizing the data is a good practice as it makes training less sensitive to scale variations in the input data. For example, in case of minimizing the mean squared error, without normalization, variables with smaller magnitude, which may be more informative in terms of predicting the output variable, may get completely overshadowed by input variables with large magnitude, but little correlation with the output variable.

XII.2. (3 points) [MID2021 Q3]

Answer True or False with justification.

- (a) A test set is redundant if you have a sufficiently large dev set.

Solution: FALSE

A test set and dev set serve different purposes. The dev set influences the choice of model hyper-parameters during training, and hence there is some bias for the dev set in the trained model. The best model is chosen based on the performance on dev set. However, the test set is completely unseen until inference time.

In an ideal case, where the train, dev and test sets are coming from the same distribution, the performance of the model on the test set should mimic the performance on the dev set (given that it is large AND diverse enough). However, this is not always the case. The distribution of the test set samples may differ from the dev set, which may lead to different performance. Further, a test set may be harder than the dev set, which can also lead to difference in performance. Hence, there is no guarantee that model performing the best on a dev set will also be the best model for a given test set.

- (b) A logistic regression model uses the logit function to handle nonlinear boundaries in the data.

Solution: FALSE

Logistic regression is a Linear classifier, with a linear decision boundary (i.e. $W^T x + b = 0$). It cannot solve for non linear boundaries. The role of the inverse logit function is to normalize the output between 0 and 1.

- (c) Gradient descent based approaches are useless for linear regression problems due to the existence of a closed form solution.

Solution: FALSE

Gradient descent is useful for linear regression problems due to its computational efficiency. The closed form solution is very easy to compute in the univariate case. However, in a general multivariate problem, we compute the solution as:-

$$\beta = (X^T X)^{-1} (X^T Y)$$

Inversion of the matrix $X^T X$ is an very expensive operation in case of problems with very high dimensionality, which are common in Machine Learning. We can use variants

of gradient descent where we keep only a part of the data in memory, lowering the requirements for computer memory.

Note: In practice, we never invert a matrix to find the closed form solution due to its cost. Instead, a system of linear equations from $(X^T X)\beta = (X^T Y)$ is solved for β . However, this is still computationally expensive and not preferred over gradient descent for large problems.

XII.3. (2 points) [MID2021 Q4]

Answer the following questions with justification.

(a) k -fold cross-validation is useful for the following.

A. Hyperparameter tuning B. Comparing classifiers C. Determining underfitting and / or overfitting D. All of the above E. None of the above

Solution: D. All of the Above.

K - fold cross validation helps in comparing models by looking at how it is performing on unseen data. Hyperparameter tuning can be done using this method. While comparing the model, we can select the one which is not overfitting.

(b) A receiver receives two types of signals, zero and non-zero. The zero signal is distributed as $p(v|y = y_z) \sim \mathcal{N}(0, 4)$, i.e., a Gaussian distribution with mean $\mu = 0$ and variance $\sigma^2 = 4$. The non-zero signal generates a voltage distributed as $p(v|y = y_{nz}) \sim 0.5\mathcal{N}(-5, 4) + 0.5\mathcal{N}(+5, 4)$. Your task is to design a classifier that, given the voltage, can identify whether the signal is zero or non-zero. Please select all that are correct from the following choices with appropriate justification.

A. A Naive Bayes classifier will serve as a good solution. B. Logistic regression may not work, but a Bayesian logistic regression (i.e., logistic regression with an appropriate regularization) will work as a good solution. C. All of the above D. None of the above

Solution: A. Naive Bayes classifier will serve as a good classifier.

Logistic regression, even with appropriate regularization, will not be a good choice as the given data is non linear. Naive Bayes on the other hand can handle this data as the data distribution is known beforehand (i.e., mixture of Gaussians).